

Departamento de Eléctrica, Electrónica y Telecomunicaciones

Luis Alexander Quishpe Mendoza

Ejercicios de programación y algoritmia

NRC: 36466

Ingeniería En Telecomunicaciones

Docente:

Ing. Jenny Alexandra Ruiz Robalino

25 de octubre del 2026

1.1 Media de dos valores

Análisis del problema

- **Entradas:** Dos números reales.
- **Proceso:** Leer ambos números, sumarlos y posteriormente dividir el resultado entre 2, guardándolo en la variable media.
- **Salida:** Mensaje indicando el resultado del proceso.

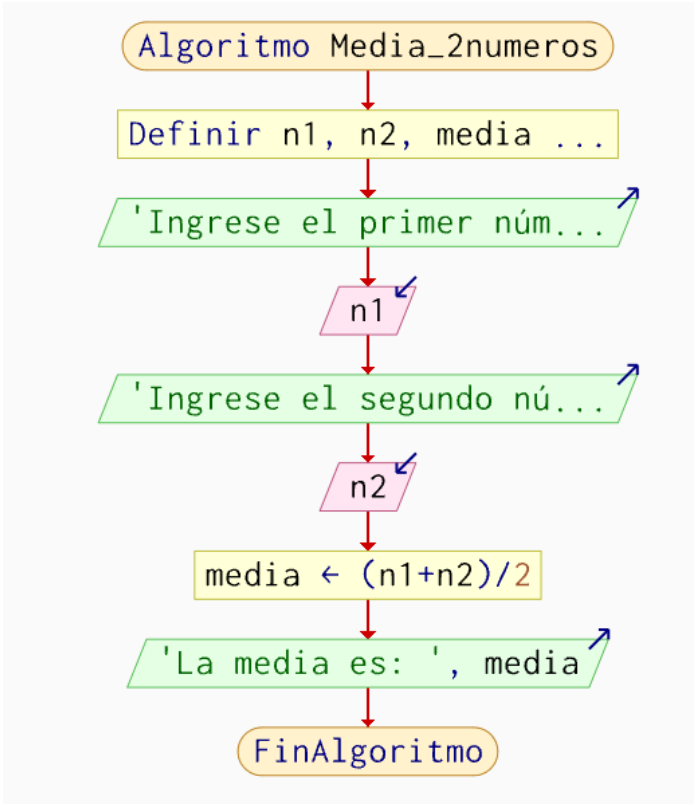
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato Entrada (#1)	n1	Var	Real
Dato Entrada (#2)	n2	Var	Real
Dato Salida (Media)	media	Var	Real
Divisor	2	Cte	Entero

Seudocódigo

```
INICIO DEL PROCESO
  Definir n1, n2, media Como Real;
  Escribir "Ingrese el primer número: ";
  Leer n1;
  Escribir "Ingrese el segundo número: ";
  Leer n2;
  media <- (n1 + n2) / 2
  Escribir "La media es: ", media;
FIN DEL PROCESO
```

Diagrama de flujo



Prueba de escritorio

Caso	n1	n2	Suma	División	Salida
Normal	10	4	10+4 = 14	14/2 = 7	La media es: 7
Resultado decimal	7	2	7+2 = 9	9/2 = 4,5	La media es: 4,5
Números iguales	10	10	10+10 = 20	20/2 = 10	La media es: 10
Un valor = 0	2	0	2+0 = 2	2/2 = 1	La media es: 1
Negativo	-9	3	-9+3 = -6	-6/2 = -3	La media es: -3
Error	"PR"	4	No se puede sumar	No se puede dividir	Error

Simulación del código

Start here | EJERCICIO 3.c

```c
#include <stdio.h>

int main() {
    float num1, num2, media;

    printf("Ingrese el primer numero: ");
    scanf("%f", &num1);

    printf("Ingrese el segundo numero: ");
    scanf("%f", &num2);

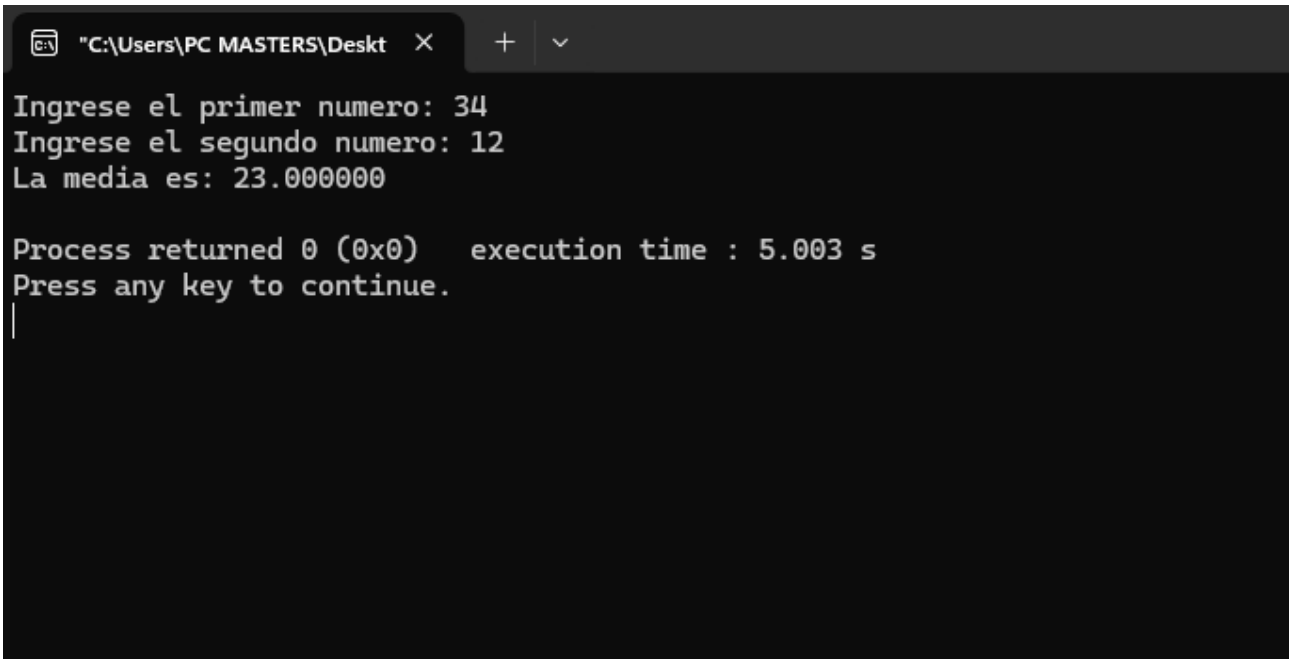
    media = (num1 + num2) / 2;

    printf("La media es: %f\n", media);
}
```

Logs & others

Code::Blocks | Search results | Cccc | Build log | Build messages | CppCheck/Vera++

```
SpellChecker: Thesaurus files 'C:\Program Files\CodeBlocks\share\codeblocks\SpellChecker\th_en_US.idx' not found!
NativeParser::DoFullParsing took: 0.354 seconds.
NativeParser::CreateParser: Finish creating a new parser for project '*NONE*'
NativeParser::OnParserEnd: Project '*NONE*' parsing stage done!
NativeParser::DeleteParser: Deleting parser for project '*NONE*'!
NativeParser::CreateParser: Finish creating a new parser for project '*NONE*'
NativeParser::OnParserEnd: Project '*NONE*' parsing stage done!
```



1.2 Valor absoluto de x al cubo

Análisis del problema

- **Entradas:** Un número real, denominado x.
- **Proceso:** Leer x, calcular el valor absoluto del número x y posteriormente elevar el resultado al cubo.
- **Salida:** Mensaje que indique el resultado del valor absoluto elevado al cubo.

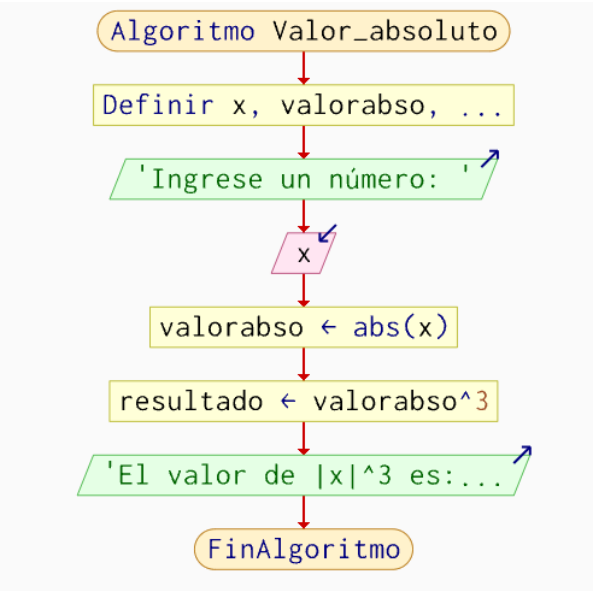
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato Entrada	x	Var	Real
Valor Absoluto	valorabso	Var	Real
Dato Salida	resultado	Var	Real
Exponente	3	Cte	Entero

Seudocódigo

```
INICIO DEL PROCESO
  Definir x, valorabso, resultado Como Real;
  Escribir "Ingrese un número: ";
  Leer x;
  valorabso <- abs(x);
  resultado <- valorabso ^ 3;
  Escribir "El valor de |x|^3 es: ", resultado;
FIN DEL PROCESO
```

Diagrama de flujo



Prueba de escritorio

Caso	x	Valor absoluto	Potencia	Salida
Normal	4	abs(4) = 4	4·4·4 = 64	El valor de |x|^3 es: 64
Negativo	-3	abs(-3) = 3	3·3·3 = 27	El valor de |x|^3 es: 27
Cero	0	abs(0) = 0	0·0·0 = 0	El valor de |x|^3 es: 0
Decimal positivo	1,5	abs(1,5) = 1,5	1,5·1,5·1,5 = 3,375	El valor de |x|^3 es: 3,375
Decimal negativo	-2,5	abs(-2,5) = 2,5	2,5·2,5·2,5 = 15,625	El valor de |x|^3 es: 15,625
Texto	"PR"	No se puede	No se puede	Error

Simulación del código

tart here | EJERCICIO 2 VALOR ABSOLUTO.c

```c
#include <stdio.h>

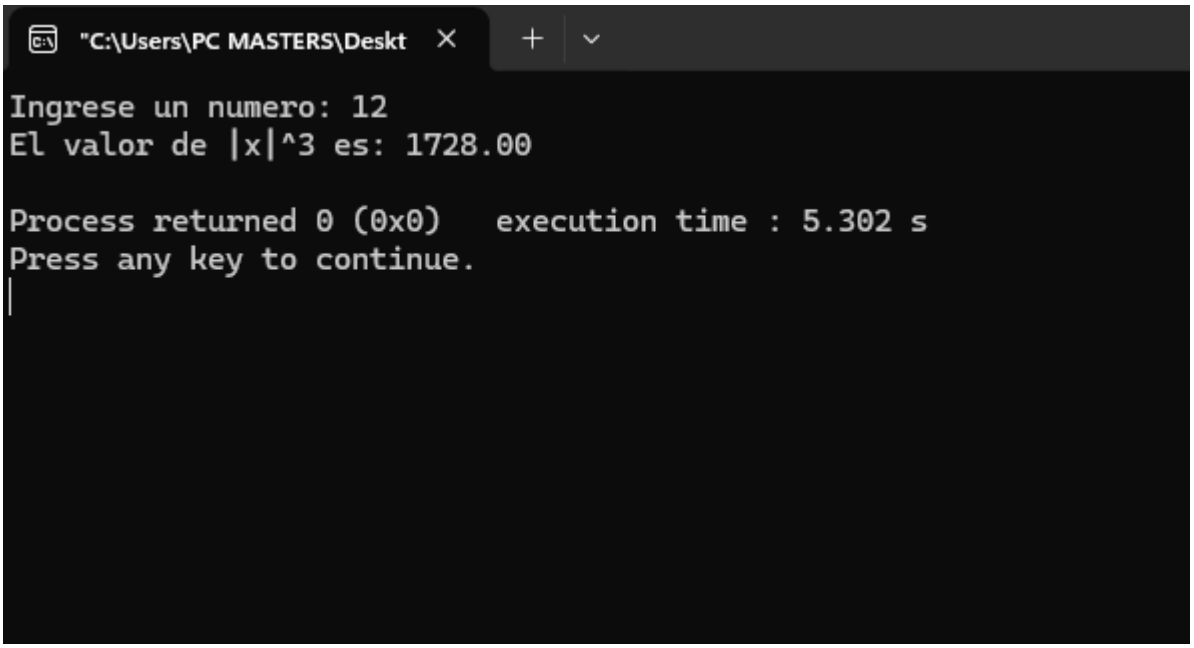
int main()
{
    double x, valorAbs, resultado;

    printf("Ingrese un numero: ");
    scanf("%lf", &x);

    if (x < 0)
        valorAbs = -x;
    else
        valorAbs = x;

    resultado = valorAbs * valorAbs * valorAbs;

    printf("El valor de |x|^3 es: %.2f\n", resultado);
    return 0;
}
```



1.3 Divisible

Análisis del problema

- **Entrada:** Dos números enteros, dividendo = x y divisor = y.
- **Proceso:** Verificar si y = 0, x es divisible por y cuando la división es entera y da cero (x MOD y = 0).
- **Salida:** Texto que indique si x es divisible por y o si no lo es.
- **Restricciones:** y ≠ 0, no existe división entre cero.1

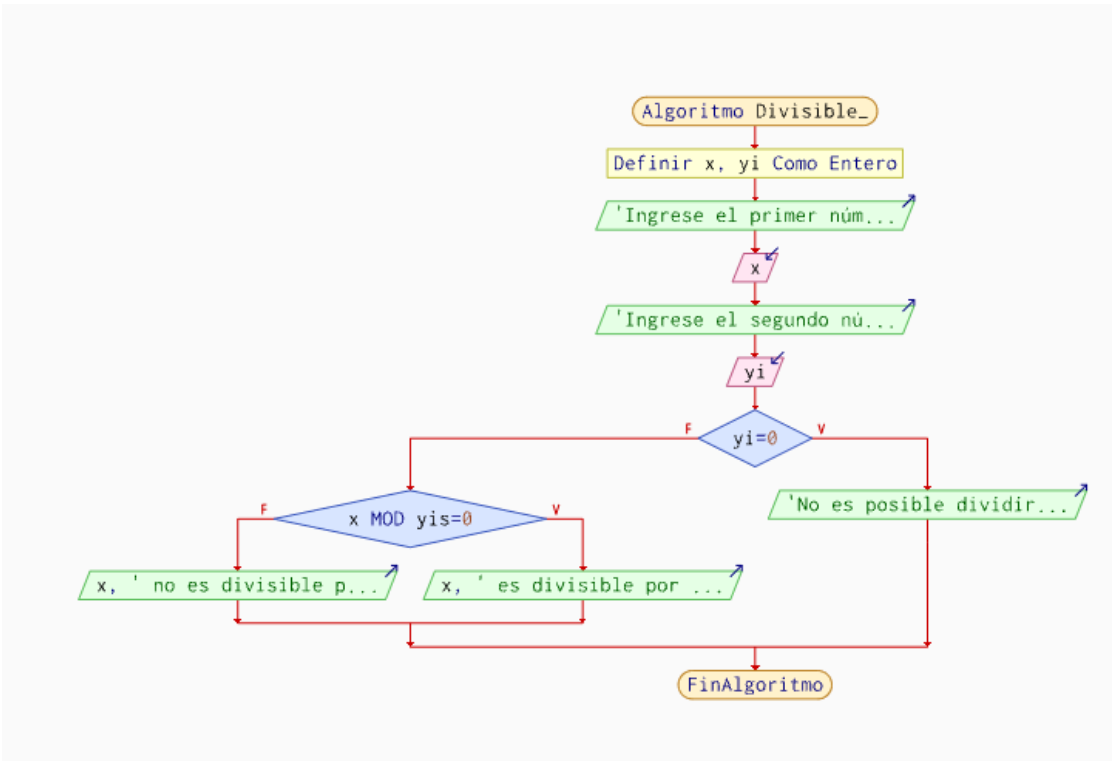
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato entrada (dividendo)	x	Var	Entero
Dato entrada (divisor)	y	Var	Entero
Cero	0	Cte	Entero

Seudocódigo

```
INICIO PROCESO
  Definir x, y Como Entero;
  Escribir "Ingrese el primer número: "
  Leer x
  Escribir "Ingrese el segundo número: "
  Leer y
  Si y = 0 Entonces
    Escribir "No es posible dividir entre cero"
  Sino
    Si x MOD y = 0 Entonces
      Escribir x, " es divisible por ", y
    Sino
      Escribir x, " no es divisible por ", y
    FinSi
  FinSi
FIN PROCESO
```

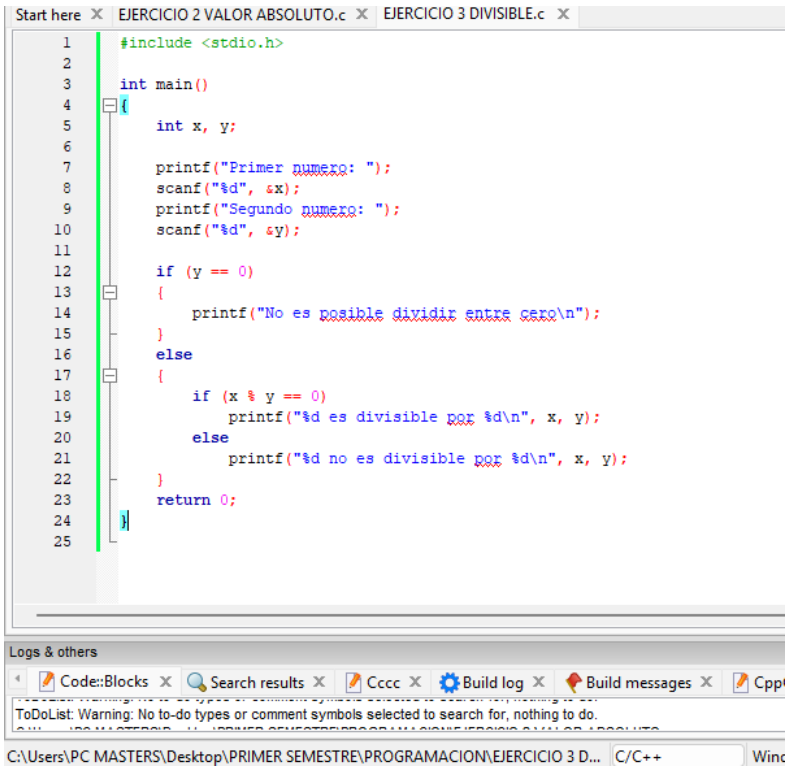
Diagrama de flujo

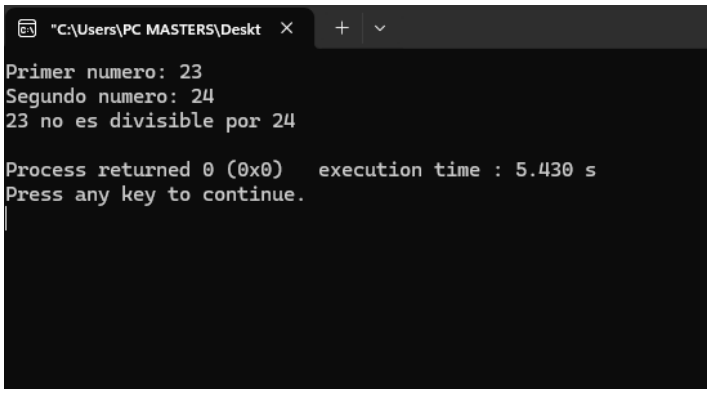


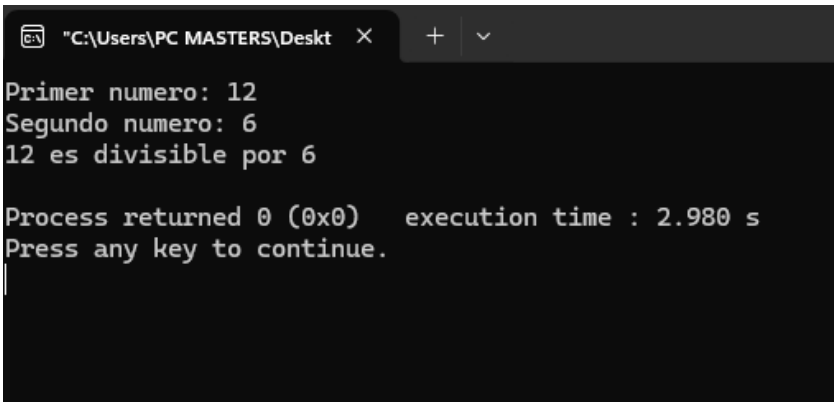
Prueba de escritorio

Caso	x	y	y = 0	x MOD y	Salida
Normal	16	8	No	0	16 es divisible por 8
Normal	8	3	No	2	8 no es divisible por 3
Iguales	2	2	No	0	2 es divisible por 2
Dividendo 0	0	4	No	0	0 es divisible por 4
Negativo	-12	4	No	0	-12 es divisible por 4
Negativo	8	-3	No	2	8 no es divisible por -3
Error (y = 0)	10	0	Sí	—	No es posible dividir entre cero

Simulación del código







1.4 Intervalo

Análisis del problema

- **Entradas:** Un número real, denominado x.
- **Proceso:** Se verifican dos condiciones en el número real: x ≥ 0 y x ≤ 10.
- **Salida:** Texto que indique si el valor pertenece al intervalo [0, 10] o si no.

Tabla de objetos

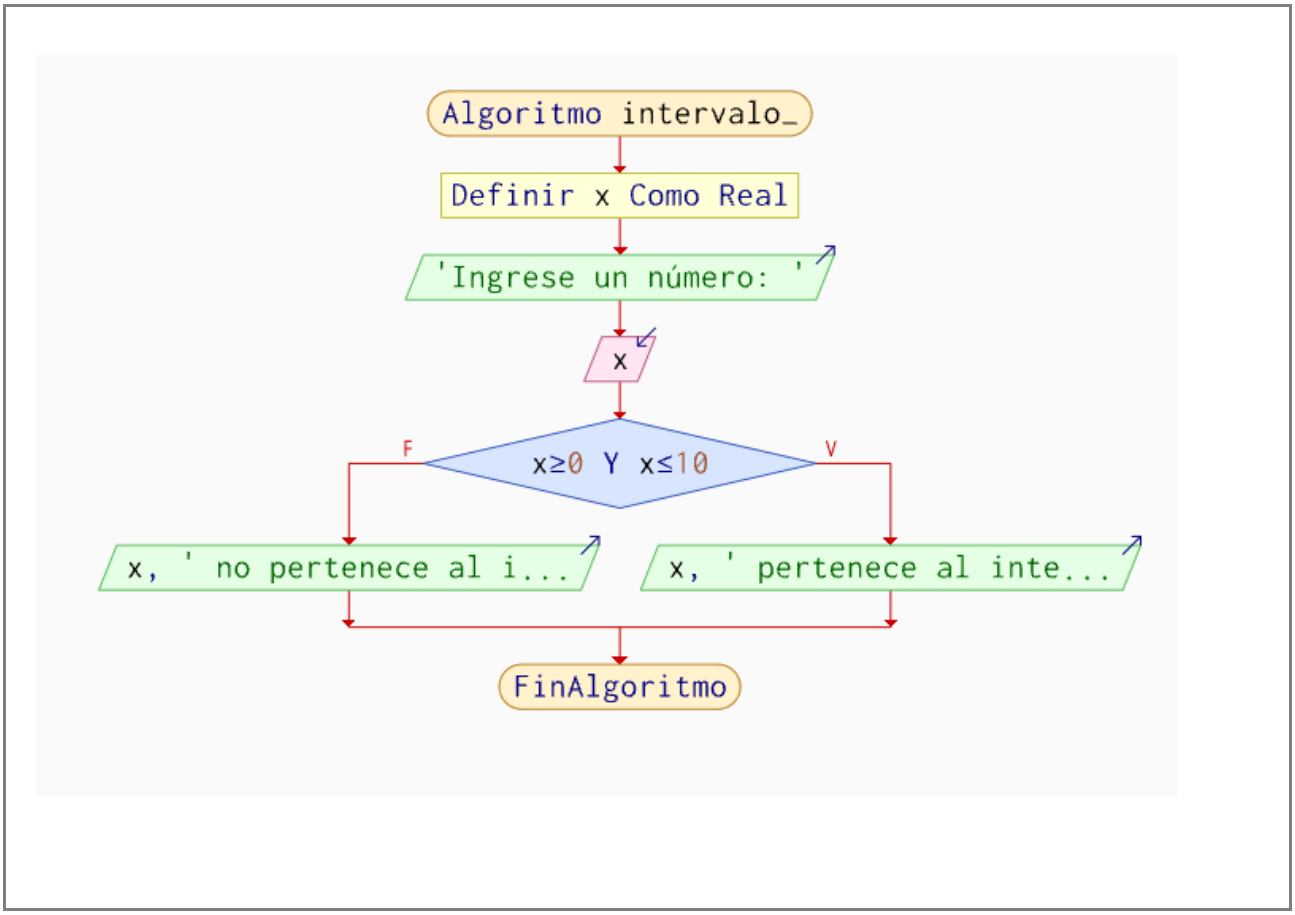
Objeto	Nombre	Valor	Tipo
Dato entrada	x	Var	Real
Límite inferior	0	Cte	Entero
Límite superior	10	Cte	Entero

Seudocódigo

```
INICIO PROCESO
  Definir x Como Real
  Escribir "Ingrese un número: "
  Leer x
  Si x >= 0 Y x <= 10 Entonces
    Escribir x, " pertenece al intervalo [0, 10]"
  Sino
    Escribir x, " no pertenece al intervalo [0, 10]"
  FinSi

FIN PROCESO
```

Diagrama de flujo



Prueba de escritorio

Caso	x	x ≥ 0	x ≤ 10	Y	Salida
Normal	6	Verdadero	Verdadero	Verdadero	6 pertenece al intervalo
Normal decimal	6,5	Verdadero	Verdadero	Verdadero	6,5 pertenece al intervalo
Límite inferior	0	Verdadero	Verdadero	Verdadero	0 pertenece al intervalo
Límite superior	10	Verdadero	Verdadero	Verdadero	10 pertenece al intervalo
Negativo	-3	Falso	Verdadero	Falso	-3 no pertenece al intervalo
Número superior	12	Verdadero	Falso	Falso	12 no pertenece al intervalo
Decimal superior	11,5	Verdadero	Falso	Falso	11,5 no pertenece al intervalo

Simulación del código

Start here | EJERCICIO 2 VALOR ABSOLUTO.c | EJERCICIO 3 DIVISIBLE.c | *EJERCICIO 4 INTERVALO.c

```c
#include <stdio.h>

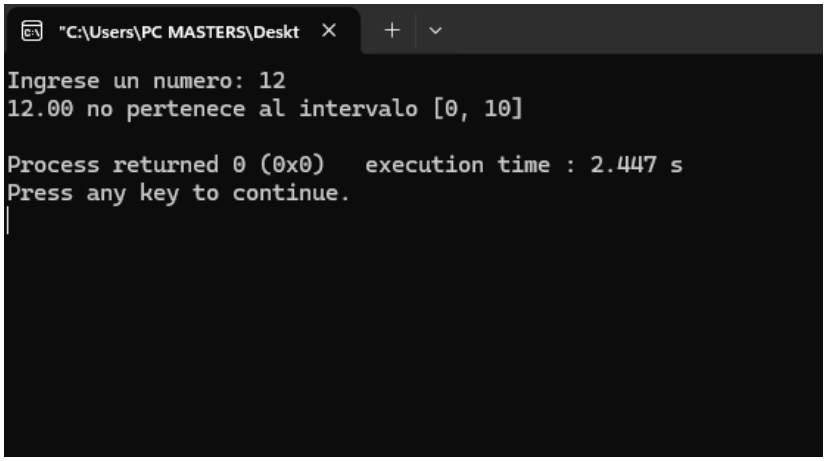
int main()
{
    double x;

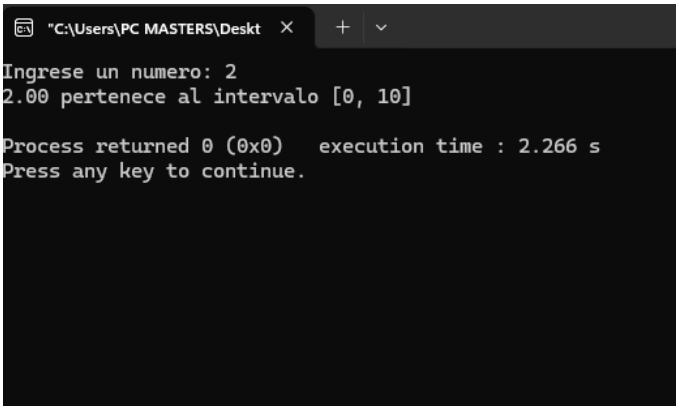
    printf("Ingrese un numero: ");
    scanf("%lf", &x);

    if (x >= 0 && x <= 10)
        printf("%.2f pertenece al intervalo [0, 10]\n", x);
    else
        printf("%.2f no pertenece al intervalo [0, 10]\n", x);
}
```

Logs & others | Code::Blocks | Search results | Cccc | Build log | Build messages | CppCheck/Vera++

C:\Users\PC MASTERS\Desktop\PRIMER SEMESTRE\PROGRAMACION\EJERCICIO 4 I... | C/C++ | Windows (CR+LF) | WINDOWS-1





1.5 Conversión de unidades de tiempo

Análisis del problema

- **Entradas:** Un número entero, indicando la duración del evento en segundos.
- **Proceso:** Mediante equivalencias fijas calculamos los días, horas y minutos en base a los segundos.
- **Salida:** Mensaje del tiempo en días, horas, minutos y segundos.
- **Restricciones:** segundos ≥ 0.

Tabla de objetos

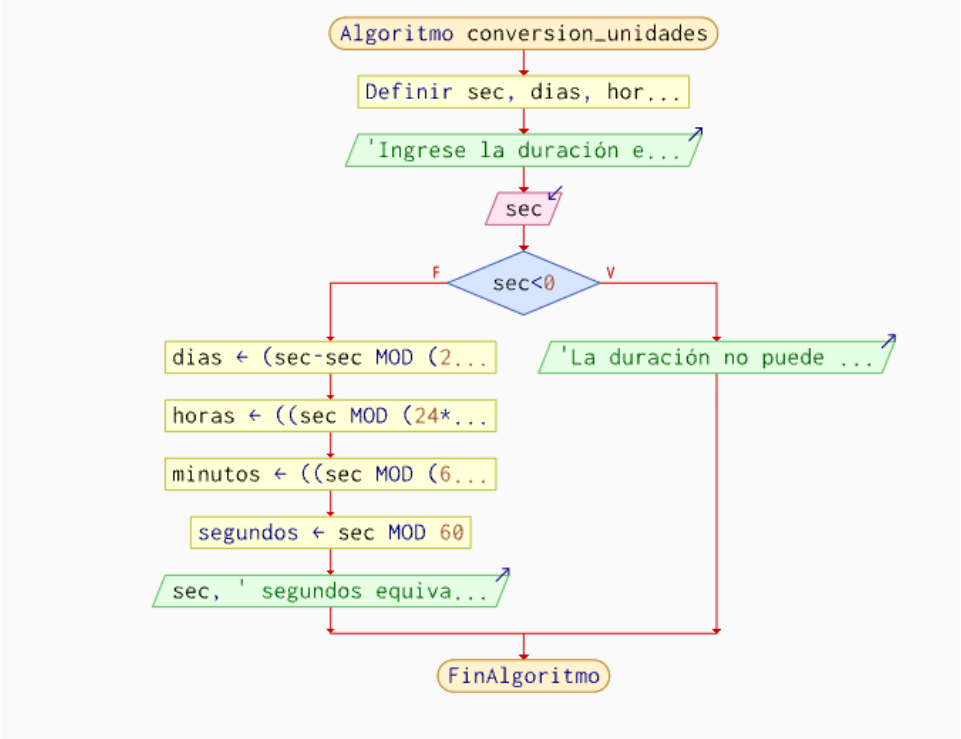
Objeto	Nombre	Valor	Tipo
Dato Entrada	sec	Var	Entero
Días	dias	Var	Entero
Horas	horas	Var	Entero
Minutos	minutos	Var	Entero
Validación	0	Cte	Entero
Segundos restantes	segundos	Var	Entero
Segundos por día	24*60*60	Cte	Entero
Segundos por hora	60*60	Cte	Entero
Segundos por minuto	60	Cte	Entero

Seudocódigo

```
INICIO PROCESO
  Definir sec, dias, horas, minutos, segundos Como Entero
  Escribir "Ingrese la duración en segundos: "
  Leer sec
  Si sec < 0 Entonces
    Escribir "La duración no puede ser negativa"
  Sino
    dias <- (sec - sec MOD (24*60*60)) / (24*60*60)
    horas <- ((sec MOD (24*60*60)) - (sec MOD (60*60))) / (60*60)
    minutos <- ((sec MOD (60*60)) - (sec MOD 60)) / 60
    segundos <- sec MOD 60
    Escribir sec, " segundos equivalen a ", dias, " días, ", horas, " horas, ", minutos, "
minutos y ", segundos, " segundos."
  FinSi

FIN PROCESO
```

Diagrama de flujo



Prueba de escritorio

Caso	sec	días	horas	min	seg	Salida
Normal	100000	1	3	46	40	100000 segundos equivalen a 1 días, 3 horas, 46 minutos y 40 segundos.
Cero	0	0	0	0	0	0 segundos equivalen a 0 días, 0 horas, 0 minutos y 0 segundos.
Límite	59	0	0	0	59	59 segundos equivalen a 0 días, 0 horas, 0 minutos y 59 segundos.
Límite	60	0	0	1	0	60 segundos equivalen a 0 días, 0 horas, 1 minutos y 0 segundos.
Límite	3600	0	1	0	0	3600 segundos equivalen a 0 días, 1 horas, 0 minutos y 0 segundos.
Límite	86400	1	0	0	0	86400 segundos equivalen a 1 días, 0 horas, 0 minutos y 0 segundos.
Negativo	-5	No existe	No existe	No existe	No existe	La duración no puede ser negativa
Texto	"PR"	No se evalúa	No se evalúa	No se evalúa	No se evalúa	No se puede evaluar

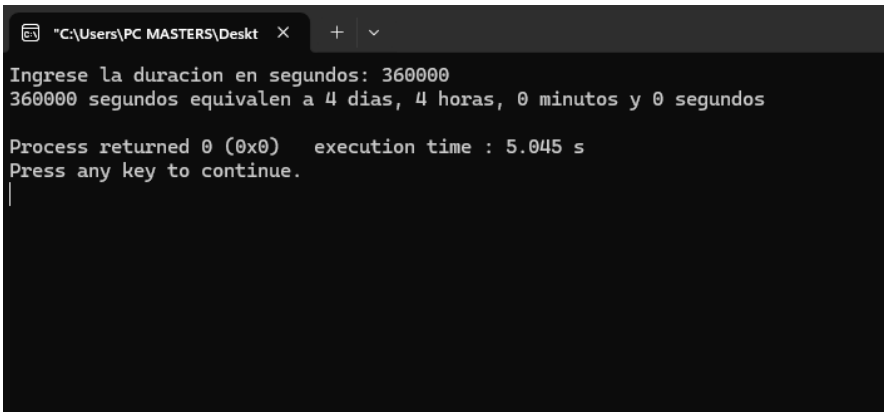
Simulación del código

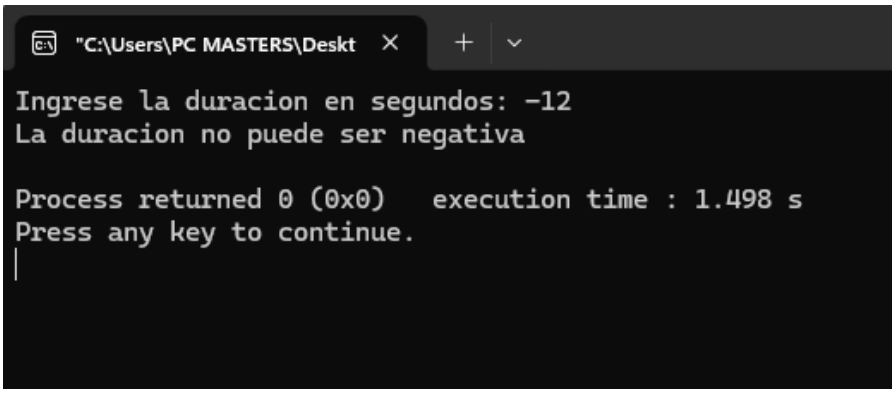
```c
int main()
{
    int c, dias, horas, minutos, segundos;

    printf("Ingrese la duracion en segundos: ");
    scanf("%d", &c);

    if (c < 0)
    {
        printf("La duracion no puede ser negativa\n");
    }
    else
    {
        /* en C, entero / entero ya corta los decimales */
        dias     = c / (24 * 60 * 60);
        horas    = (c % (24 * 60 * 60)) / (60 * 60);
        minutos  = (c % (60 * 60)) / 60;
        segundos = c % 60;

        printf("%d segundos equivalen a %d dias, %d horas, %d minutos y %d segundos\n",
               c, dias, horas, minutos, segundos);
    }
}
```





1.6 Máximo, mínimo y media de alturas

Análisis del problema

- **Entradas:** Un valor real, que se lea varias veces, hasta que se ingrese un valor negativo.
- **Proceso:** Almacenar datos hasta que exista un negativo. Se preparan valores iniciales tal que: suma = 0, cantidad = 0, y máximo y mínimos iguales a la primera altura. Posteriormente se suman las cantidades y se actualizan el máximo y el mínimo dependiendo de si ingresa un valor mayor al máximo o menor al mínimo. Por último, se calcula la media: suma / cantidad.
- **Salidas:** Datos sobre la media, el máximo y el mínimo.

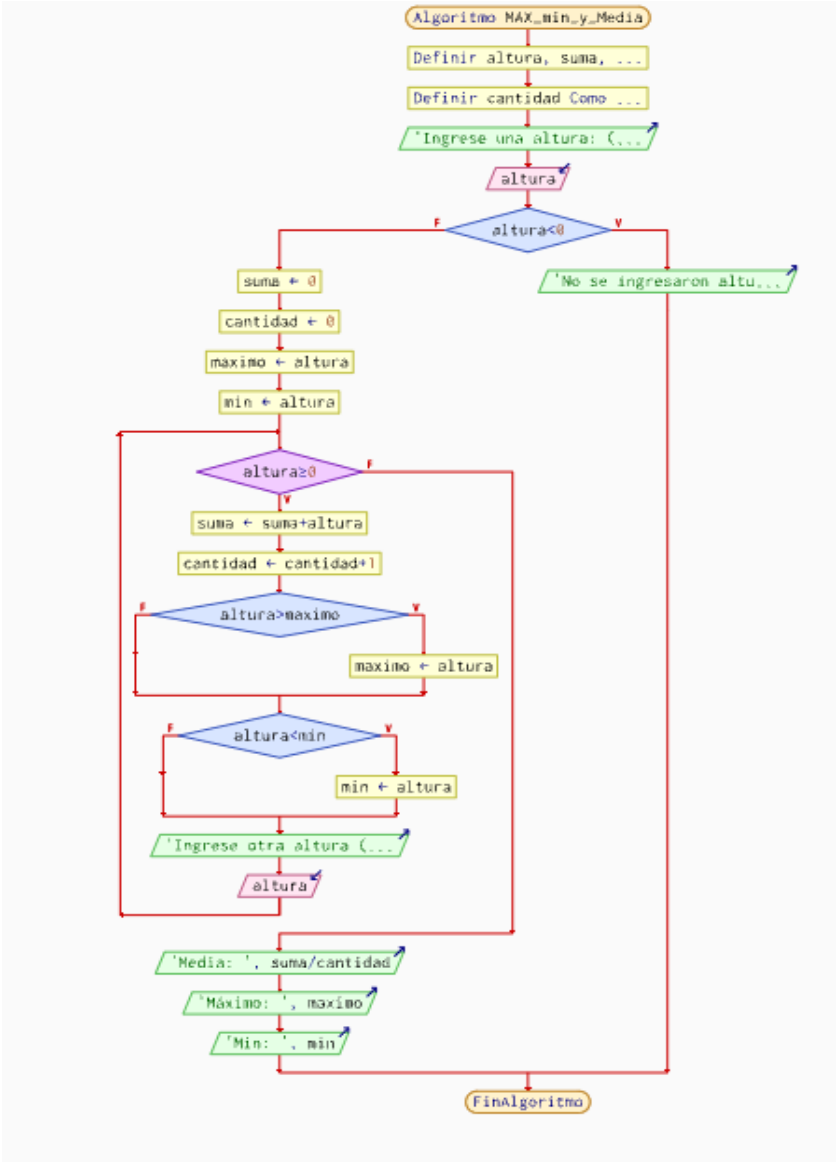
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato Entrada	altura	Var	Real
Suma Alturas	suma	Var	Real
Cantidad Alturas	cantidad	Var	Entero
Altura máxima	maximo	Var	Real
Altura mínima	min	Var	Real
Media	media	Var	Real
Valor inicial	0	Cte	Entero
Incremento	1	Cte	Entero

Seudocódigo

```
INICIAR PROCESO
  Definir altura, suma, maximo, min Como Real
  Definir cantidad Como Entero
  Escribir "Ingrese una altura: (termina con un número negativo)"
  Leer altura
  Si altura < 0 Entonces
    Escribir "No se ingresaron alturas"
  Sino
    suma <- 0
    cantidad <- 0
    maximo <- altura
    min <- altura
    Mientras altura >= 0 Hacer
      suma <- suma + altura
      cantidad <- cantidad + 1
      Si altura > maximo Entonces
        maximo <- altura
      FinSi
      Si altura < min Entonces
        min <- altura
      FinSi
      Escribir "Ingrese otra altura (un número negativo termina): "
      Leer altura
    FinMientras
    Escribir "Media: ", suma / cantidad
    Escribir "Máximo: ", maximo
    Escribir "Min: ", min
  FinSi
FIN PROCESO
```

Diagrama de flujo



Prueba de escritorio

Caso	Alturas ingresadas	Proceso	Salida
Normal	1,80 / 1,90 / 1,70 / -1	suma = 5,4 / cantidad = 3 / máx = 1,90 / mín = 1,70 / media = 5,4/3	Media: 1,8 / Máximo: 1,90 / Min: 1,70
1 dato	1,75 / -1	suma = 1,75 / cantidad = 1 / máx = 1,75 / mín = 1,75	Media: 1,75 / máximo = mínimo
Iguales	1,90 / 1,90 / -1	suma = 3,8 / cantidad = 2 / máx = 1,90 / mín = 1,90	Media: 1,90 / máximo = mínimo
Máx y mín	1,90 / 1,50 / 1,40 / -1	suma = 4,8 / cantidad = 3 / máx = 1,90 / mín = 1,40	Media: 1,60 / Máximo: 1,90 / Min: 1,40
Con 0	0 / 1,50 / -1	suma = 1,50 / cantidad = 2 / máx = 1,50 / mín = 0	Media: 0,75 / Máximo: 1,50 / Min: 0
Negativo	-1	No entra al ciclo	No se ingresaron alturas
Texto	"PR"	No se evalúa	No se puede evaluar

Simulación del código

```
#include <stdio.h>

int main()
{
   double altura, suma = 0, maximo, minimo, media;
   int cantidad = 0;

   printf("Altura (negativa para salir): ");
   scanf("%lf", &altura);

   if (altura >= 0)
   {
      maximo = altura;
      minimo = altura;

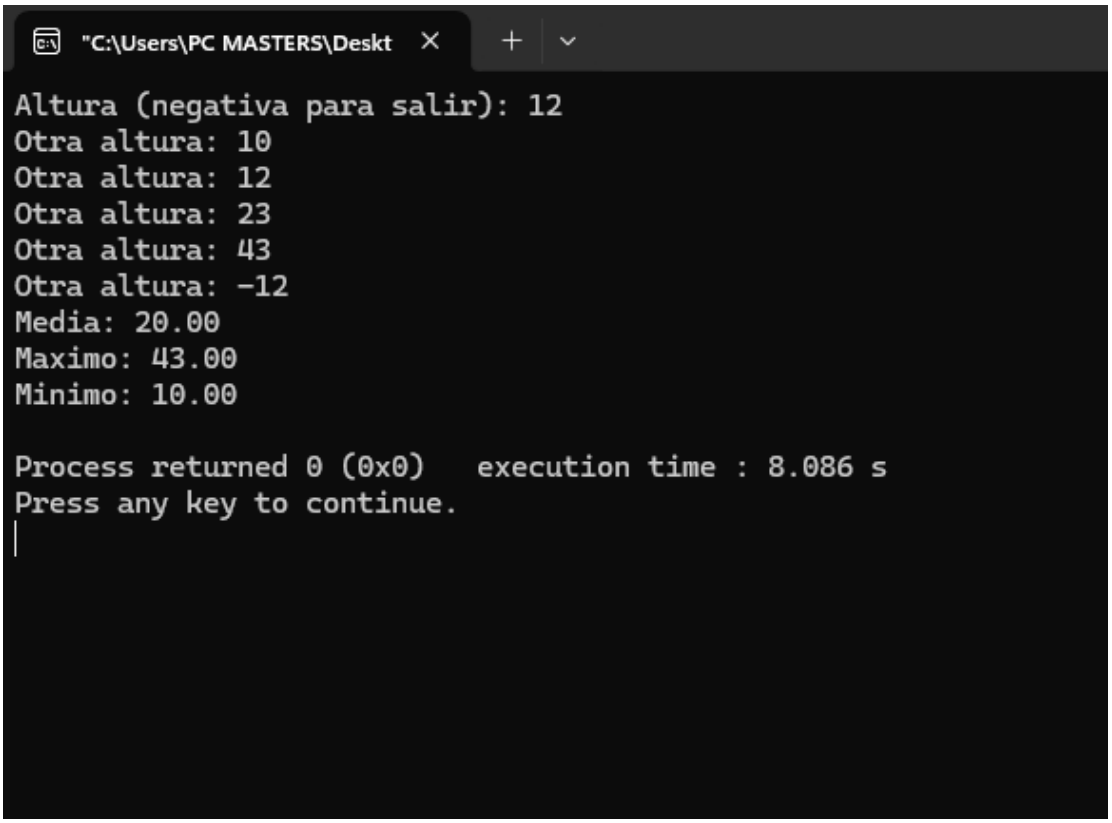
      while (altura >= 0)
      {
         suma = suma + altura;
         cantidad++;

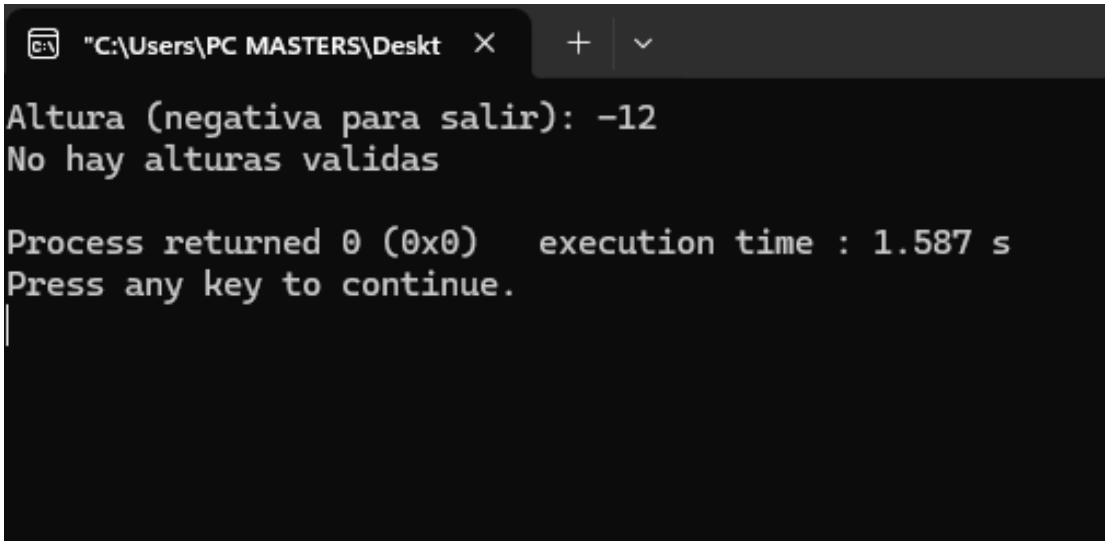
         if (altura > maximo)
            maximo = altura;
         if (altura < minimo)
            minimo = altura;

         printf("Otra altura: ");
         scanf("%lf", &altura);
      }

      media = suma / cantidad;
      printf("Media: %.2f\n", media);
      printf("Maximo: %.2f\n", maximo);
      printf("Minimo: %.2f\n", minimo);
   }
   else
   {
      printf("No hay alturas validas\n");
   }

}
```


"C:\Users\PC MASTERS\Deskt
Altura (negativa para salir): 12
Otra altura: 10
Otra altura: 12
Otra altura: 23
Otra altura: 43
Otra altura: -12
Media: 20.00
Maximo: 43.00
Minimo: 10.00
Process returned 0 (0x0)   execution time : 8.086 s
Press any key to continue.


"C:\Users\PC MASTERS\Deskt
Altura (negativa para salir): -12
No hay alturas validas
Process returned 0 (0x0)   execution time : 1.587 s
Press any key to continue.

1.7 Raíz cuadrada entera

Análisis del problema

- **Entradas:** Un valor entero llamado número.
- **Proceso:** Leer N y verificar si es positivo. Empezamos con i = 1 y usamos el comando Mientras i*i ≤ N, aumentar i en 1.
- **Salida:** Resultado de la raíz cuadrada entera.

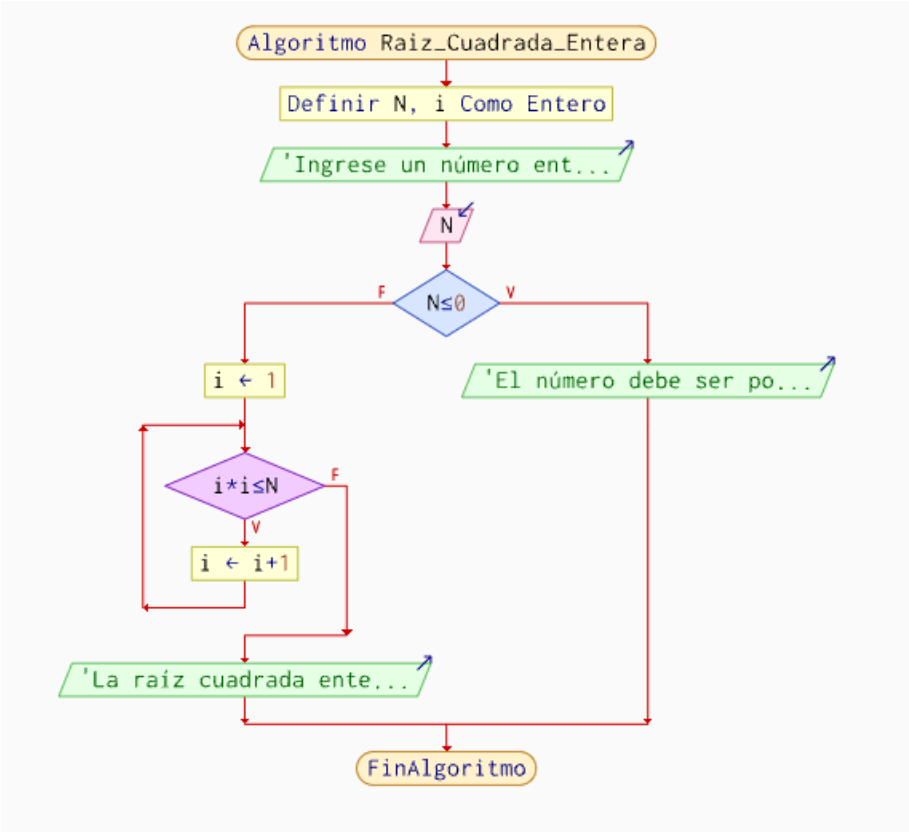
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato Entrada	N	Var	Entero
Candidato Raíz	i	Var	Entero
Valor inicial	1	Cte	Entero
Límite validación	0	Cte	Entero

Seudocódigo

```
INICIO PROCESO
  Definir N, i Como Entero
  Escribir "Ingrese un número entero positivo: "
  Leer N
  Si N <= 0 Entonces
    Escribir "El número debe ser positivo"
  Sino
    i <- 1
    Mientras i * i <= N Hacer
      i <- i + 1
    FinMientras
    Escribir "La raíz cuadrada entera de ", N, " es: ", i - 1
  FinSi
FIN PROCESO
```

Diagrama de flujo



Prueba de escritorio

Caso	Entrada	Proceso	Salida
Normal	2	$1^2 = 1 \leq 2 \rightarrow i = 2$; $2^2 = 4 \leq 2$ (No). i = 2; R = 2 - 1	La raíz cuadrada entera de 2 es: 1
Perfecto	25	i = 1 … ($5^2 = 25 \leq 25$) → i = 6; $6^2 = 36 \leq 25$ (No); R = 6 - 1	La raíz cuadrada entera de 25 es: 5
Cero	0	N ≤ 0 es verdadero	El número debe ser positivo
Negativo	-9	N ≤ 0 es verdadero	El número debe ser positivo

Simulación del código

```c
#include <stdio.h>

int main()
{
    int N, i;

    printf("Ingrese un entero positivo: ");
    scanf("%d", &N);

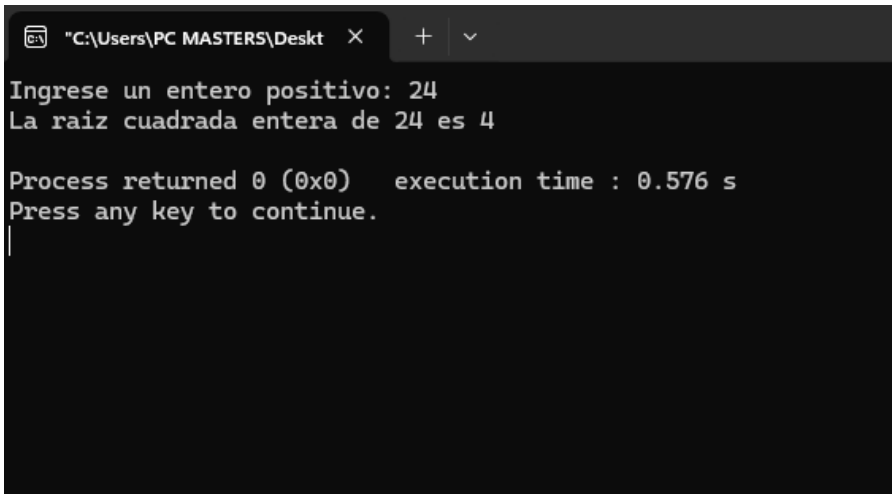
    if (N <= 0)
    {
        printf("El numero debe ser positivo\n");
    }
    else
    {
        i = 1;
        while (i * i <= N)
        {
            i++;
        }
        printf("La raiz cuadrada entera de %d es %d\n", N, i - 1);
    }

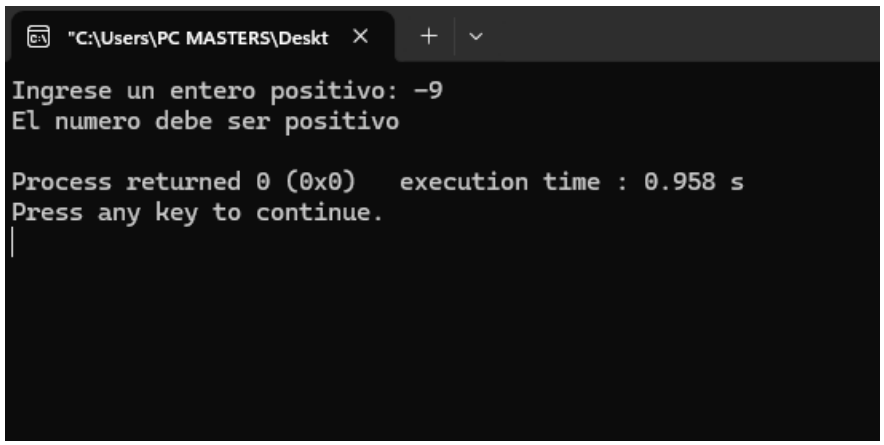
}
```

Logs & others

Code::Blocks | Search results | Cccc | Build log | Build messages | CppCheck/Vera++

C:\Users\PC MASTERS\Desktop\PRIMER SEMESTRE\PROGRAMACION\EJERCICIO 7 RAIZ CUADRADA.c





1.8 Número primo

Análisis del problema

- **Entradas:** Un número entero, denominado N.
- **Proceso:** Verificar que el número N > 1. Suponemos que es primo y empezamos a comprobar con i = 2; usamos el comando Mientras i < N: si N MOD i = 0, se encontró un divisor, por ende no es primo (primo = Falso); procedemos con i = i + 1.
- **Salida:** Mensaje indicando si el número es primo o no.
- **Restricciones:** Número mayor que 1; el 2 es primo (único par).

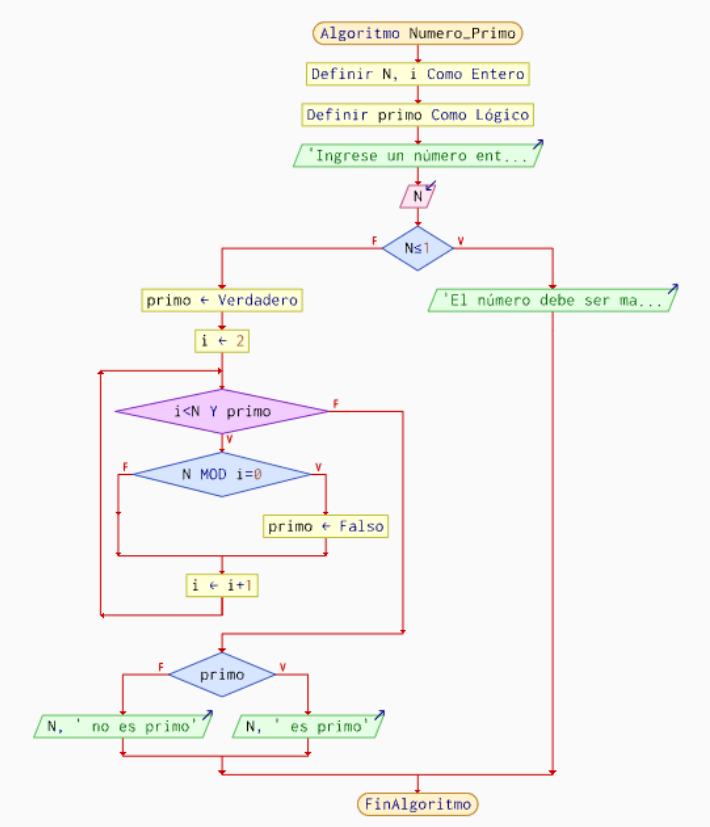
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato Entrada	N	Var	Entero
Divisor Candidato	i	Var	Entero
Indicador	primo	Var	Lógico
1.er divisor a comprobar	2	Cte	Entero
Límite	1	Cte	Entero
Incremento	1	Cte	Entero
Comparación	0	Cte	Entero

Seudocódigo

```
INICIO PROCESO
  Definir N, i Como Entero
  Definir primo Como Logico
  Escribir "Ingrese un número entero mayor que 1: "
  Leer N
  Si N <= 1 Entonces
    Escribir "El número debe ser mayor que 1"
  Sino
    primo <- Verdadero
    i <- 2
    Mientras i < N Y primo Hacer
      Si N MOD i = 0 Entonces
        primo <- Falso
      FinSi
      i <- i + 1
    FinMientras
    Si primo Entonces
      Escribir N, " es primo"
    Sino
      Escribir N, " no es primo"
    FinSi
  FinSi
FIN PROCESO
```

Diagrama de flujo



Prueba de escritorio

Caso	N	Proceso	Salida
Normal	13	i = 2 hasta 12: restos 1, 1, 1, 3, 1, 6, 5, 4, 3, 2, 1 (ninguno es 0); al terminar i = 13 y primo = V	13 es primo
Normal compuesto	21	i = 2: 21 MOD 2 = 1. i = 3: 21 MOD 3 = 0 → primo = F	21 no es primo
Par	18	i = 2: 18 MOD 2 = 0 → primo = F	18 no es primo
Cuadrado	49	i = 2, 3, 4, 5, 6, 7: 49 MOD 7 = 0 → primo = F	49 no es primo
Menor	2	2 < 2 es Falso (no entra al ciclo); primo = V	2 es primo
Error	1	N ≤ 1 → Verdadero	El número debe ser mayor que 1
Texto	"PR"	No se puede evaluar	No se puede evaluar

Simulación del código

```
#include <stdio.h>

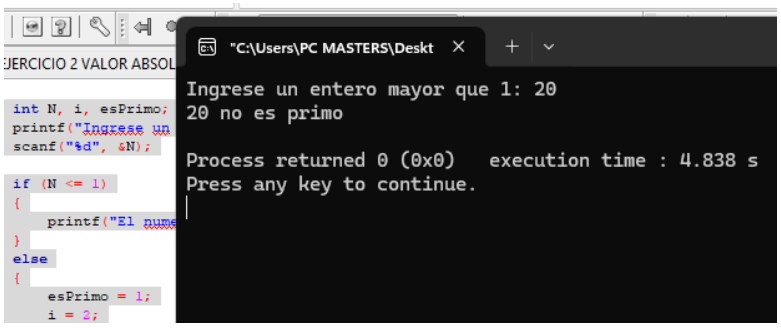
int main()
{
   int N, i, esPrimo;
   printf("Ingrese un entero mayor que 1: ");
   scanf("%d", &N);

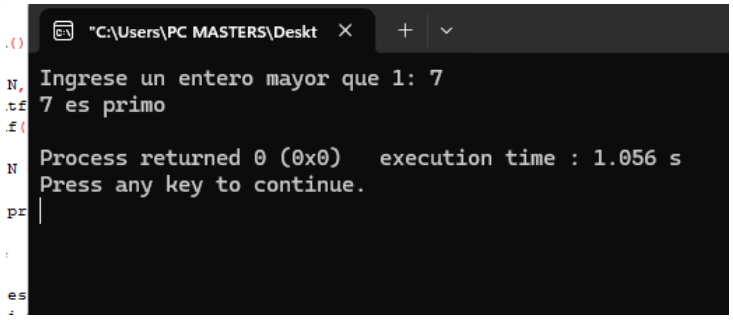
   if (N <= 1)
   {
      printf("El numero debe ser mayor que 1\n");
   }
   else
   {
      esPrimo = 1;
      i = 2;

      while (i < N && esPrimo)
      {
         if (N % i == 0)
            esPrimo = 0;
         i++;
      }

      if (esPrimo)
         printf("%d es primo\n", N);
      else
         printf("%d no es primo\n", N);
   }

}
```





1.9 Números primos comprendidos entre dos números dados

Análisis del problema

- **Entradas:** Dos valores enteros, denominados (a, b).
- **Procesos:** Verificar que a ≤ b y realizar un ciclo externo que corra desde el número a hasta b. Para cada número aplicar la prueba del 1.8: se supone primo = V y se prueba con i = 2; mientras i < n buscar un divisor con n MOD i = 0.
- **Salidas:** Texto indicando cada primo del rango, mostrado en orden creciente.
- **Restricciones:** a > b → se avisa y no se calcula.

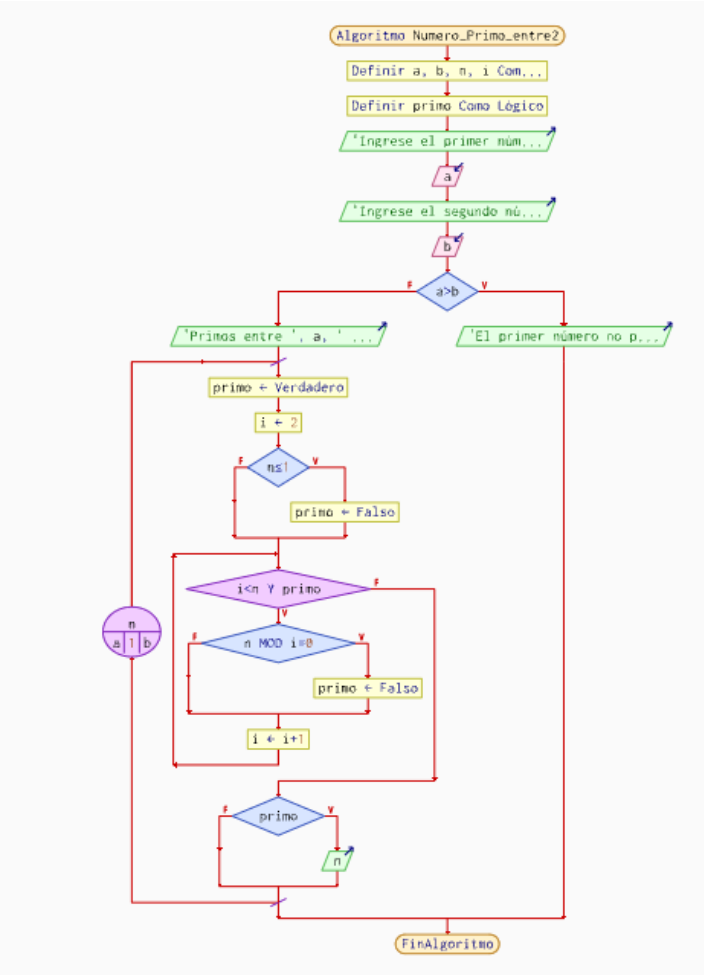
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	a	Var	Entero
Dato de entrada	b	Var	Entero
Dato de entrada	n	Var	Entero
Divisor Candidato	i	Var	Entero
Primo	primo	Var	Lógico
1.er divisor a probar	2	Cte	Entero
Límite primos	1	Cte	Entero
Incremento	1	Cte	Entero
Comparación	0	Cte	Entero

Seudocódigo

```
INICIO PROCESO
  Definir a, b, n, i Como Entero
  Definir primo Como Logico
  Escribir "Ingrese el primer número entero: "
  Leer a
  Escribir "Ingrese el segundo número entero: "
  Leer b
  Si a > b Entonces
    Escribir "El primer número no puede ser mayor que el segundo"
  Sino
    Escribir "Primos entre ", a, " y ", b, ":"
    Para n <- a Hasta b Con Paso 1 Hacer
      primo <- Verdadero
      i <- 2
      Si n <= 1 Entonces
        primo <- Falso
      FinSi
      Mientras i < n Y primo Hacer
        Si n MOD i = 0 Entonces
          primo <- Falso
        FinSi
        i <- i + 1
      FinMientras
      Si primo Entonces
        Escribir n
      FinSi
    FinPara
  FinSi
FIN PROCESO
```

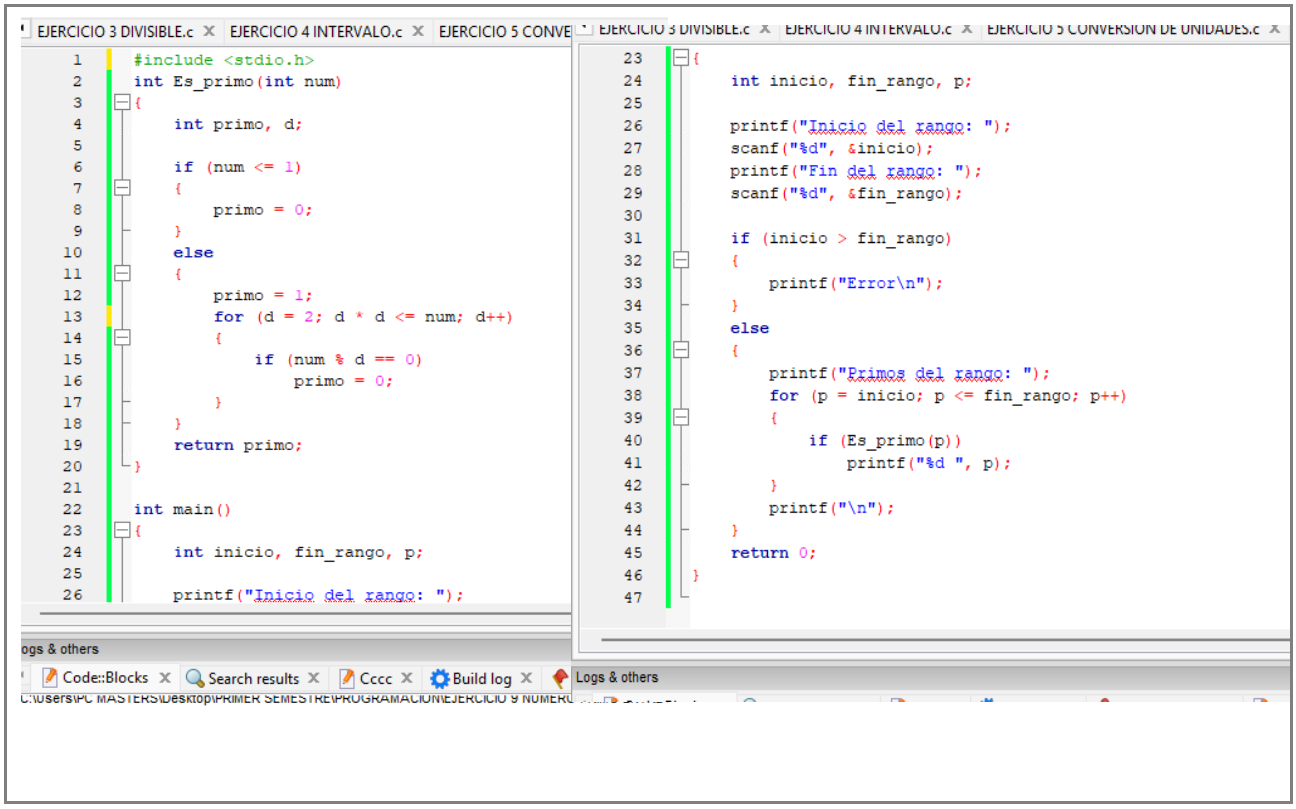
Diagrama de flujo



Prueba de escritorio

Caso	a, b	Proceso	Salida
Normal	20, 30	20 (÷2), 21 (÷3), 22 (÷2), 23 (primo), 24 (÷2), 25 (÷5), 26 (÷2), 27 (÷3), 28 (÷2), 29 (primo), 30 (÷2)	23, 29
Rango con 0	0, 12	0 y 1 descartados (n ≤ 1); 2, 3, 4 (÷2), 5, 6 (÷2), 7, 8 (÷2), 9 (÷3), 10 (÷2), 11, 12 (÷2)	2, 3, 5, 7, 11
Negativos	-3, 8	-3, -2, -1, 0 y 1 descartados; 2, 3, 4 (÷2), 5, 6 (÷2), 7, 8 (÷2)	2, 3, 5, 7
Límites primos	11, 13	11, 12 (÷2), 13	11, 13
Límite 1 primo	7, 7	n = 7: no hay divisor entre 2 y 6	7
Límite 1 no primo	9, 9	n = 9: 9 MOD 3 = 0	No mostrar nada
Límite sin primos	32, 36	32 (÷2), 33 (÷3), 34 (÷2), 35 (÷5), 36 (÷2)	No mostrar nada
Error	30, 20	a > b → Verdadero	El primer número no puede ser mayor que el segundo
Texto	"xy", 5	No se puede evaluar	Error: dato no cumple

Simulación del código



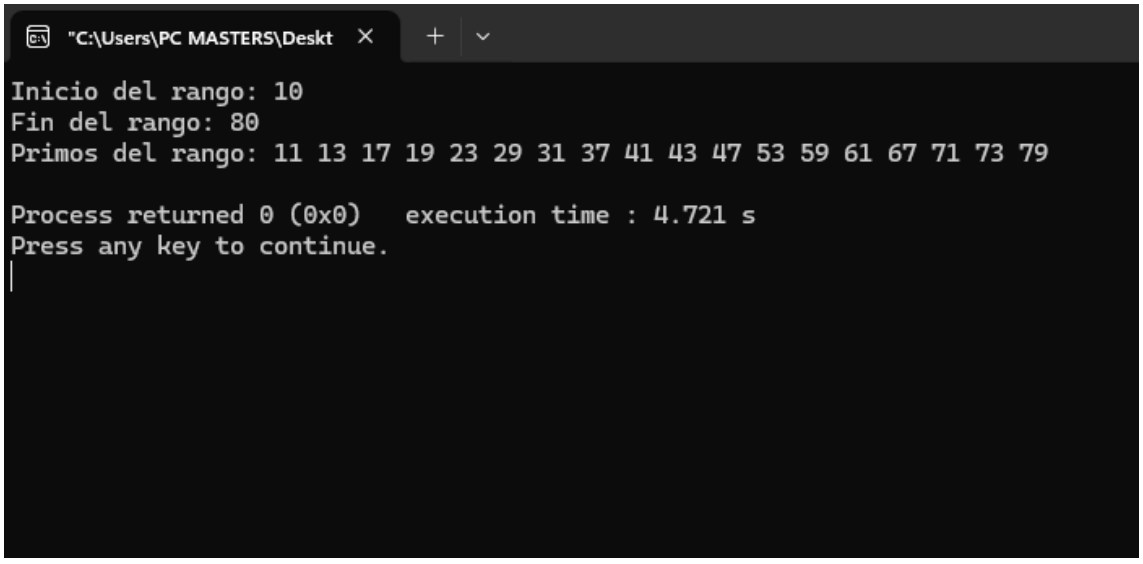
```c
#include <stdio.h>
int Es_primo(int num)
{
    int primo, d;

    if (num <= 1)
    {
        primo = 0;
    }
    else
    {
        primo = 1;
        for (d = 2; d * d <= num; d++)
        {
            if (num % d == 0)
                primo = 0;
        }
    }
    return primo;
}

int main()
{
    int inicio, fin_rango, p;

    printf("Inicio del rango: ");
    scanf("%d", &inicio);
    printf("Fin del rango: ");
    scanf("%d", &fin_rango);

    if (inicio > fin_rango)
    {
        printf("Error\n");
    }
    else
    {
        printf("Primos del rango: ");
        for (p = inicio; p <= fin_rango; p++)
        {
            if (Es_primo(p))
                printf("%d ", p);
        }
        printf("\n");
    }
    return 0;
}
```



```
"C:\Users\PC MASTERS\Deskt
Inicio del rango: 10
Fin del rango: 80
Primos del rango: 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79

Process returned 0 (0x0)   execution time : 4.721 s
Press any key to continue.
```

1.10 Factorial

Análisis del problema

- **Entradas:** Un número entero que indica un valor mayor o igual que 0, que se escribe desde el teclado del usuario.
- **Proceso:** Verificamos que el número cumpla con n ≥ 0; posteriormente preparamos el acumulador factorial = 1 y lo multiplicamos por i.
- **Salida:** Texto indicando el valor de n!, tomando el valor del factorial.
- **Restricción:** n debe ser mayor o igual a cero.

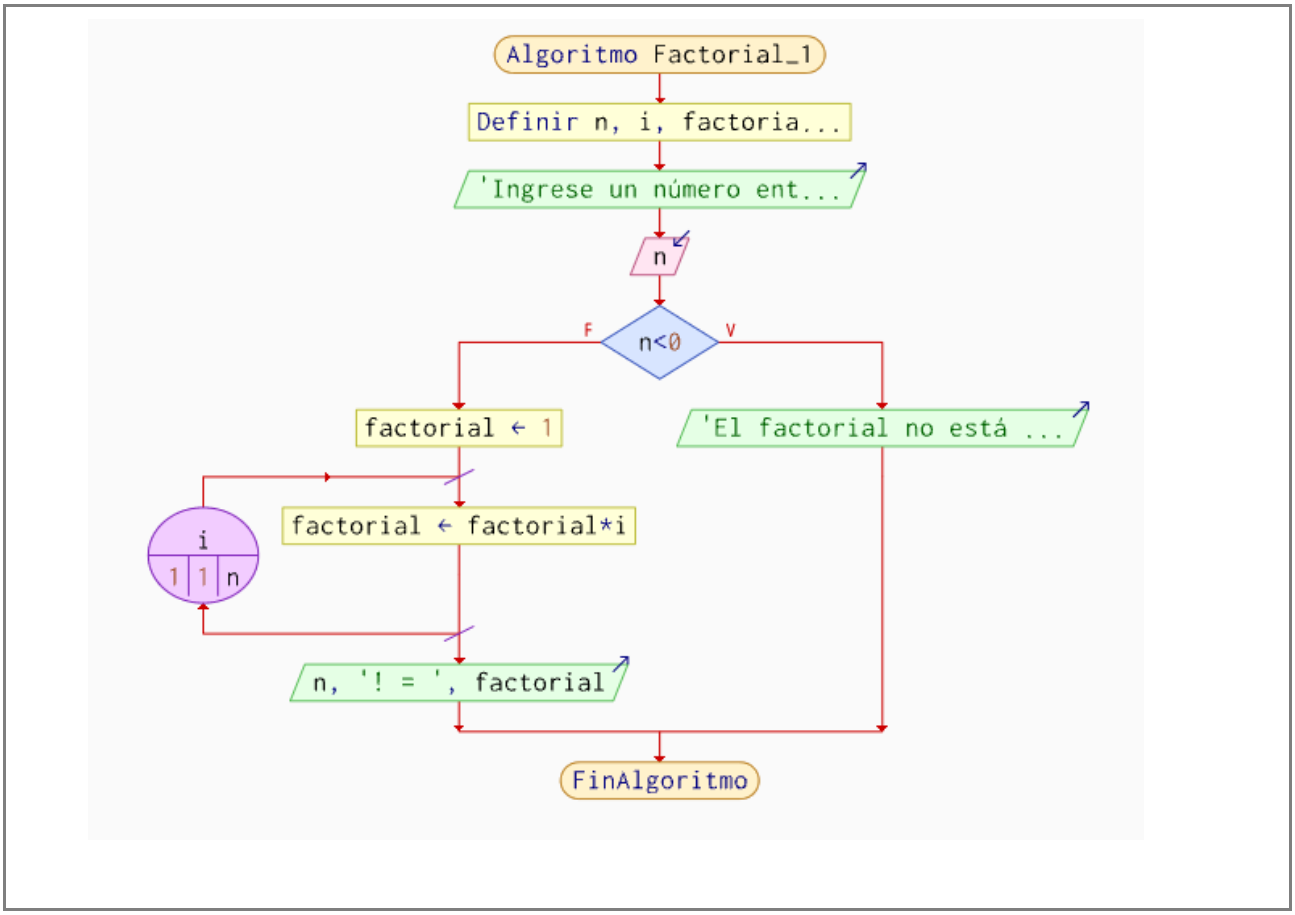
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de Entrada	n	Var	Entero
Contador de ciclo	i	Var	Entero
Factorial	factorial	Var	Entero
Valor inicial	1	Cte	Entero
Validación	0	Cte	Entero

Seudocódigo

```
INICIO PROCESO
  Definir n, i, factorial Como Entero
  Escribir "Ingrese un número entero mayor o igual que 0: "
  Leer n
  Si n < 0 Entonces
    Escribir "El factorial no está definido para números negativos"
  Sino
    factorial <- 1
    Para i <- 1 Hasta n Con Paso 1 Hacer
      factorial <- factorial * i
    FinPara
    Escribir n, "! = ", factorial
  FinSi
FIN PROCESO
```

Diagrama de flujo



Prueba de escritorio

Caso	n	Proceso	Salida
Ejemplo guía	5	1·1 = 1; 1·2 = 2; 2·3 = 6; 6·4 = 24; 24·5 = 120	5! = 120
Normal	4	1·1 = 1; 1·2 = 2; 2·3 = 6; 6·4 = 24	4! = 24
Normal (mayor)	9	8! = 40320; 40320·9 = 362880	9! = 362880
Límite cero	0	i <- 1 Hasta 0: el ciclo no se ejecuta, factorial = 1	0! = 1
Borde int 32 bits	12	12! = 479001600, el mayor factorial que entra	12! = 479001600
Desborde	13	13! = 6227020800, mayor que el límite de 32 bits	Valor incorrecto
Negativo	-5	n < 0 → Verdadero	Factorial no definido para negativos
Texto	xyz	No se puede evaluar	Error

Simulación del código

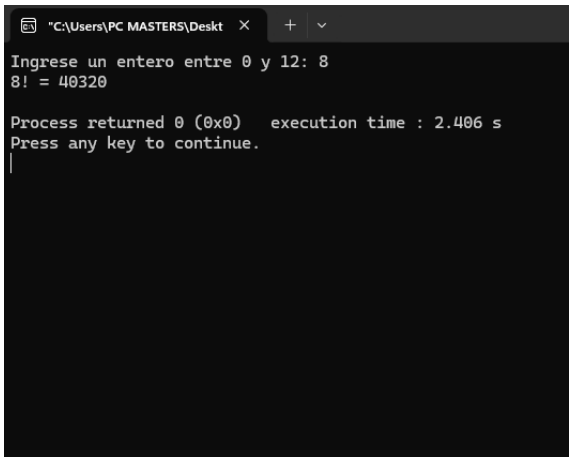
```c
#include <stdio.h>

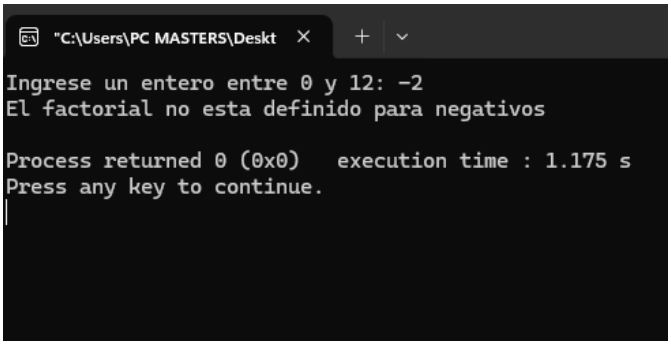
int main()
{
    int n, i, factorial;

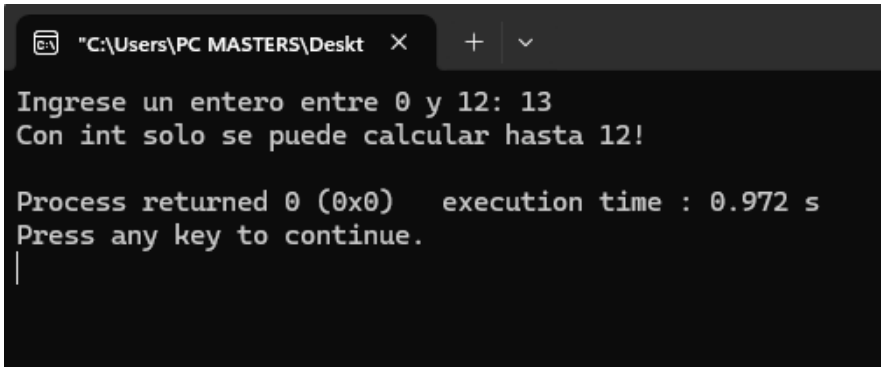
    printf("Ingrese un entero entre 0 y 12: ");
    scanf("%d", &n);

    if (n < 0)
    {
        printf("El factorial no esta definido para negativos\n");
    }
    else if (n > 12)
    {
        printf("Con int solo se puede calcular hasta 12!\n");
    }
    else
    {
        factorial = 1;
        for (i = 1; i <= n; i++)
        {
            factorial = factorial * i;
        }
        printf("%d! = %d\n", n, factorial);
    }
    return 0;
}
```

Logs & others | Code::Blocks | Search results | Cccc | Build log | Build messages







1.11 Serie de Fibonacci

Análisis del problema

- **Entrada:** Un número entero, llamado n, que nos indica cuántos términos se van a mostrar.
- **Proceso:** Hacer uso de un ciclo que se repita n veces; por cada una mostrar el valor del término anterior, posteriormente se calcula el término siguiente sumando y se actualizan los valores para la siguiente vuelta.
- **Salida:** Mensaje indicando la secuencia de los n términos de Fibonacci.

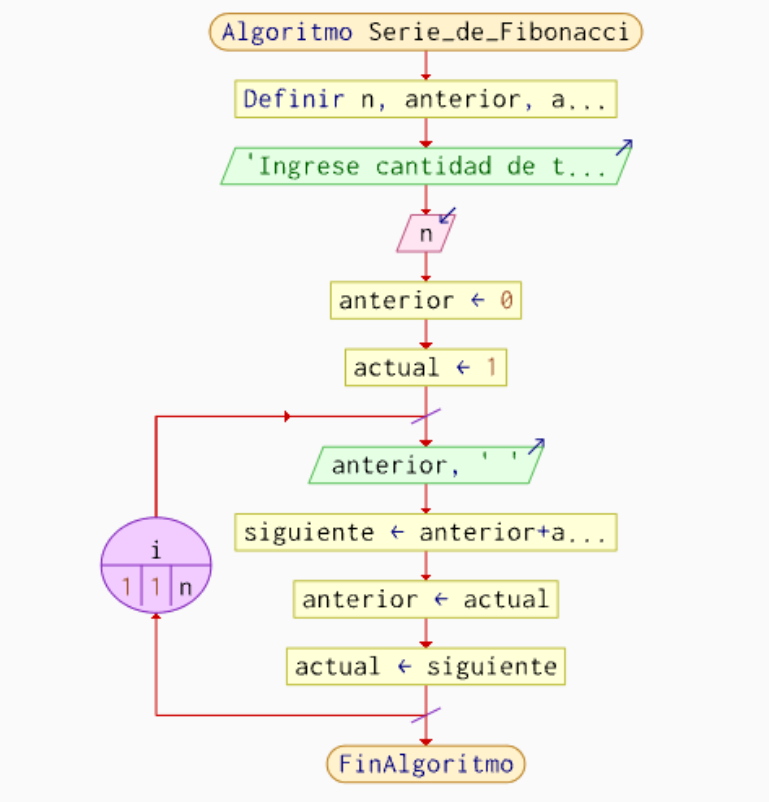
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Entrada cantidad #	n	Leer / Var	Entero
# Anterior de la secuencia	anterior	0 / Cte	Entero
# Actual de la secuencia	actual	1 / Cte	Entero
# Siguiente de la secuencia	siguiente	0+1 /	Entero
Contador	i	1 / Cte	Entero

Seudocódigo

```
INICIO ALGORITMO SERIE FIBONACCI
  Definir n, anterior, actual, siguiente, i Como Entero
  Escribir "Ingrese cantidad de términos (n >= 1): "
  Leer n
  anterior <- 0
  actual <- 1
  Para i <- 1 Hasta n Con Paso 1 Hacer
    Escribir anterior, " "
    siguiente <- anterior + actual
    anterior <- actual
    actual <- siguiente
  FinPara
FIN ALGORITMO
```

Diagrama de flujo



Prueba de escritorio

Paso (i)	Proceso	anterior	siguiente	Nuevo anterior	Nuevo actual
Inicio	n = 9	0	—	—	1
1	1 ≤ 9 (Sí)	0	0+1 = 1	1	1
2	2 ≤ 9 (Sí)	1	1+1 = 2	1	2
3	3 ≤ 9 (Sí)	1	1+2 = 3	2	3
4	4 ≤ 9 (Sí)	2	2+3 = 5	3	5
5	5 ≤ 9 (Sí)	3	3+5 = 8	5	8
6	6 ≤ 9 (Sí)	5	5+8 = 13	8	13
7	7 ≤ 9 (Sí)	8	8+13 = 21	13	21
8	8 ≤ 9 (Sí)	13	13+21 = 34	21	34
9	9 ≤ 9 (Sí)	21	21+34 = 55	34	55
10	10 ≤ 9 (No)	FIN	—	—	—

Salida en pantalla: 0 1 1 2 3 5 8 13 21

Simulación del código

Start here | *EJERCICIO 10 FACTORIAL.c | *EJERCICIO 11 FIBONACCI.c

```c
#include <stdio.h>

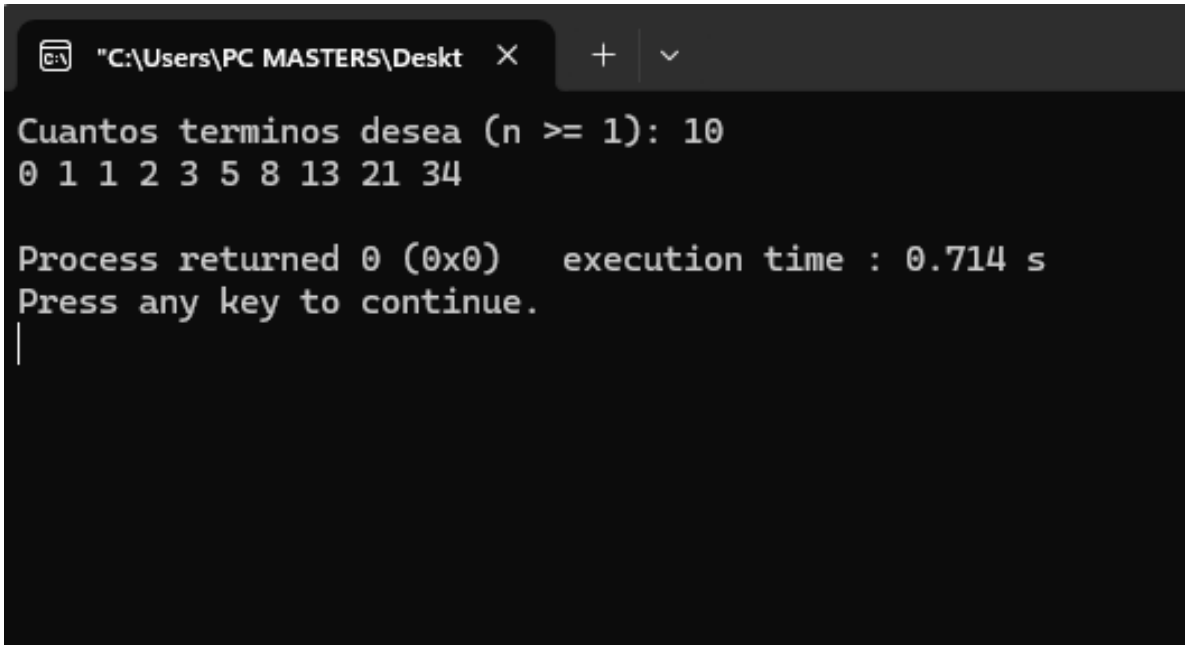
int main()
{
    int n, i, anterior, actual, siguiente;

    printf("Cuantos terminos desea (n >= 1): ");
    scanf("%d", &n);

    if (n < 1)
    {
        printf("n debe ser mayor o igual que 1\n");
    }
    else
    {
        anterior = 0;
        actual = 1;

        for (i = 1; i <= n; i++)
        {
            printf("%d ", anterior);
            siguiente = anterior + actual;
            anterior = actual;
            actual = siguiente;
        }
        printf("\n");
    }
}
```

Logs & others | Code::Blocks | Search results | Cccc | Build log | Build mes



1.12 Suma de dígitos

Análisis del problema

- **Entrada:** Un entero positivo, denominado num.
- **Proceso:** Evaluamos el número para ver si es mayor que 0; si lo es, mediante el módulo (MOD) extraemos su último dígito y lo sumamos a un acumulador. Posteriormente realizamos una división para eliminar el último dígito del número. Se repite x veces para el número de x dígitos.
- **Salida:** Mensaje indicando el resultado de la variable suma, que contiene el total de los dígitos acumulados.

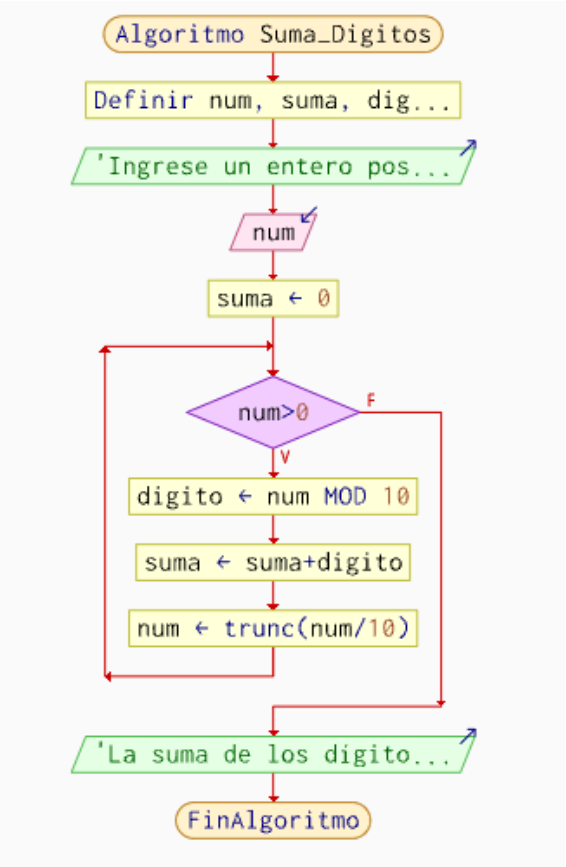
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Entrada # entero	num	Var	Entero
Acumulador / suma	suma	0	Entero
Dígito extraído	digito	0	Entero

Seudocódigo

```
INICIO ALGORITMO SUMA DÍGITOS
  Definir num, suma, digito Como Entero
  Escribir "Ingrese un entero positivo: "
  Leer num
  suma <- 0
  Mientras num > 0 Hacer
    digito <- num MOD 10
    suma <- suma + digito
    num <- trunc(num / 10)
  FinMientras
  Escribir "La suma de los dígitos es: ", suma
FIN ALGORITMO
```

Diagrama de flujo



Prueba de escritorio

Paso	Condición	digito	suma	Salida (num)
Inicio	—	—	—	2425
1	2425 > 0 (Sí)	2425 MOD 10 = 5	0+5 = 5	trunc(2425/10) = 242
2	242 > 0 (Sí)	242 MOD 10 = 2	5+2 = 7	trunc(242/10) = 24
3	24 > 0 (Sí)	24 MOD 10 = 4	7+4 = 11	trunc(24/10) = 2
4	2 > 0 (Sí)	2 MOD 10 = 2	11+2 = 13	trunc(2/10) = 0
Fin	0 > 0 (No)	—	—	La suma de los dígitos es: 13

Simulación del código

```c
#include <stdio.h>
int main()
{
    int n, suma;

    printf("Ingrese un entero positivo: ");
    scanf("%d", &n);

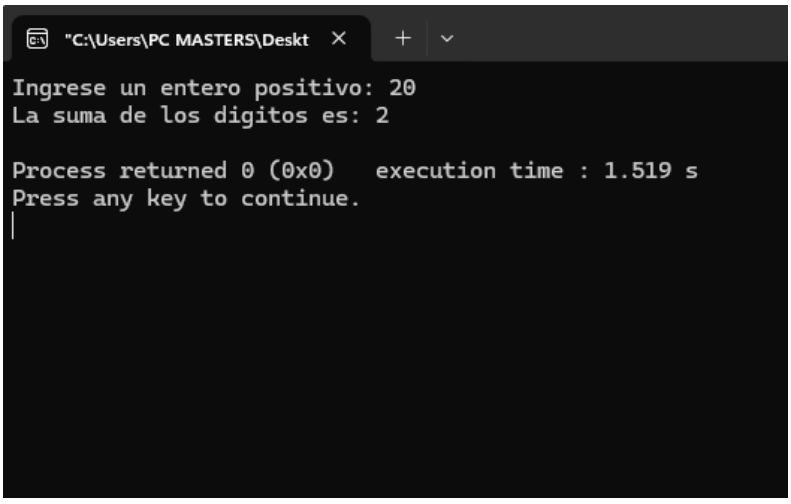
    if (n <= 0)
    {
        printf("El numero debe ser positivo\n");
    }
    else
    {
        suma = 0;
        while (n > 0)
        {
            suma = suma + (n % 10);
            n = n / 10;
        }
        printf("La suma de los digitos es: %d\n", suma);
    }

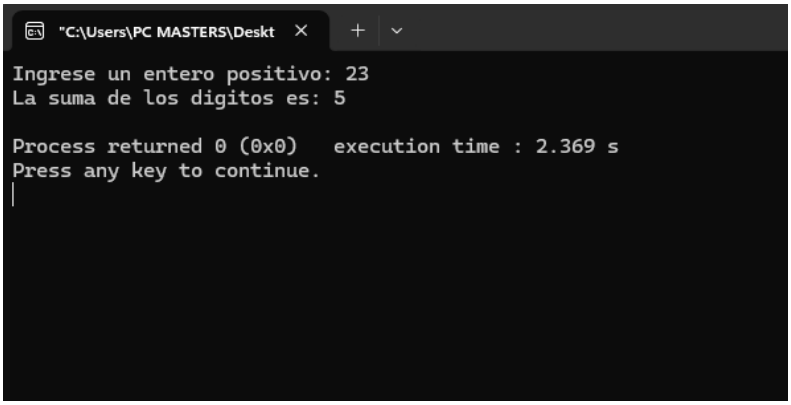
}
```

Logs & others

Code::Blocks | Search results | Cccc | Build log | Build messages

C:\Users\PC MASTERS\Desktop\PRIMER SEMESTRE\PROGRAMACION\EJERCICIO 12 SUMA DE DIGITOS.c





1.13 Número capicúa

Análisis del problema

- **Entrada:** Un número entero, denominado num.
- **Proceso:** Almacenamos el valor original de num en una variable para posteriormente descomponerlo sin perder el num original. Analizamos los últimos dígitos, los extraemos e invertimos su posición.
- **Salida:** Mensaje indicando si el número cumple con la condición de capicúa o no.

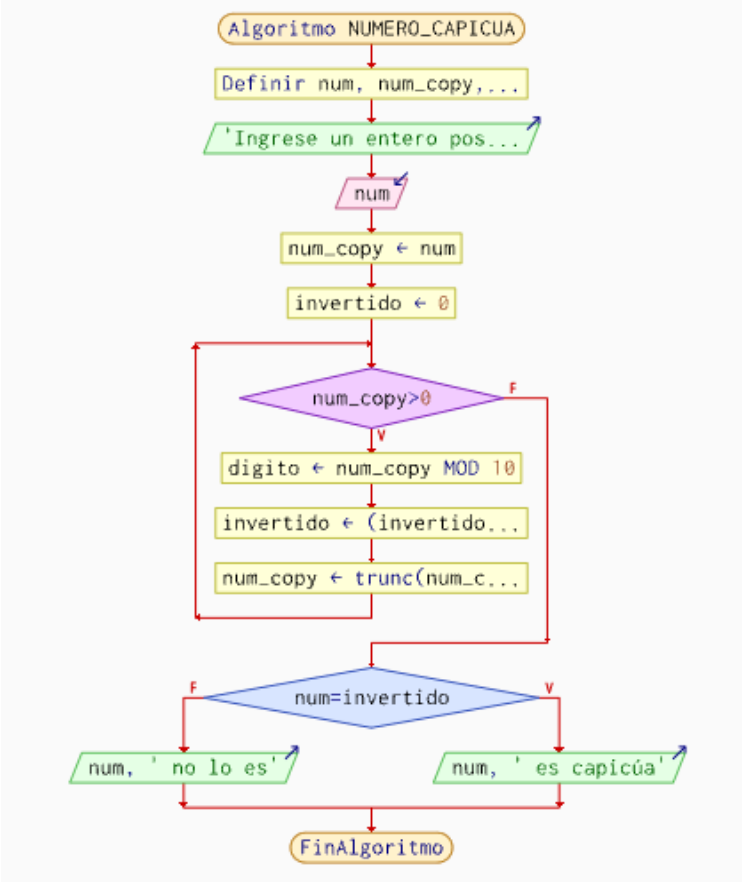
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Entrada # entero	num	Var	Entero
Copia # entero	num_copy	num	Entero
Var # invertido	invertido	0	Entero
Dígito extraído	digito	0	Entero

Seudocódigo

```
INICIO ALGORITMO NÚMERO CAPICÚA
  Definir num, num_copy, invertido, digito Como Entero
  Escribir "Ingrese un entero positivo: "
  Leer num
  num_copy <- num
  invertido <- 0
  Mientras num_copy > 0 Hacer
    digito <- num_copy MOD 10
    invertido <- (invertido * 10) + digito
    num_copy <- trunc(num_copy / 10)
  FinMientras
  Si num = invertido Entonces
    Escribir num, " es capicúa"
  Sino
    Escribir num, " no lo es"
  FinSi
FIN ALGORITMO
```

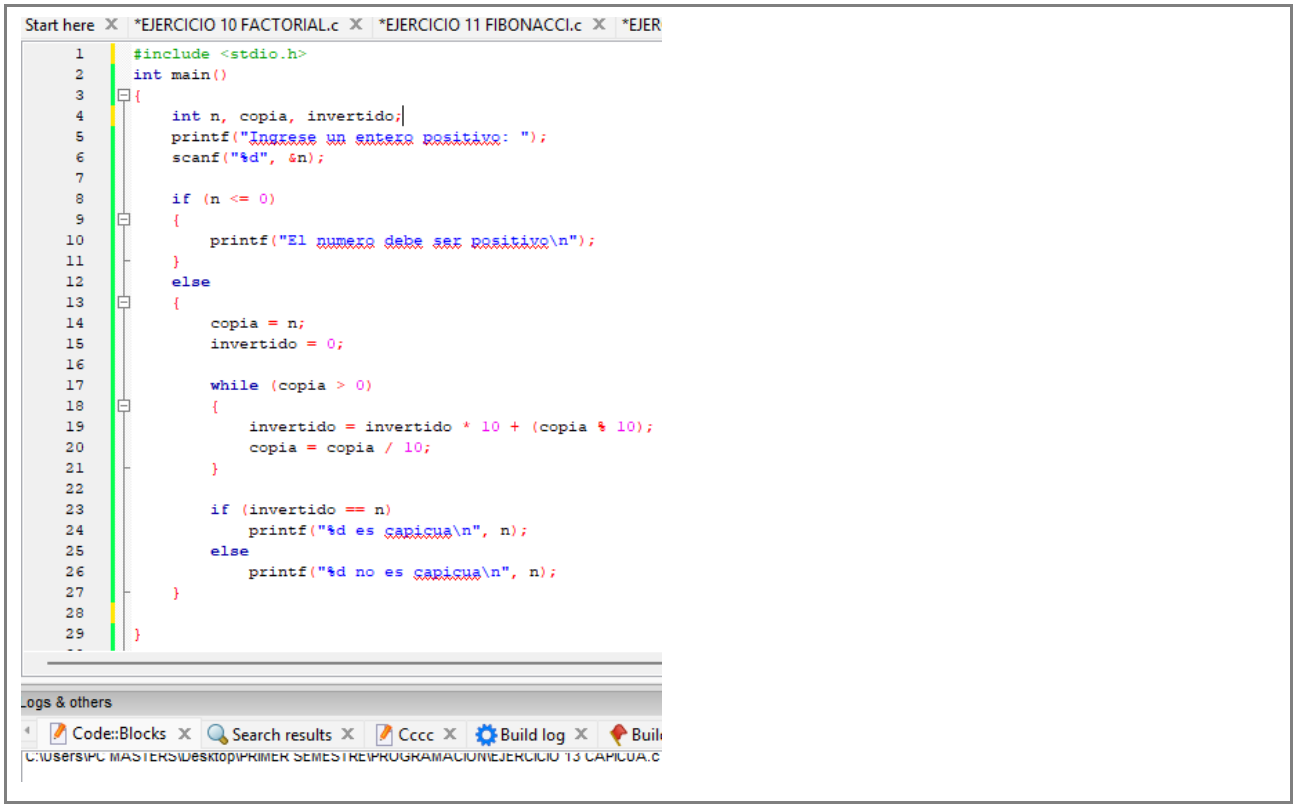
Diagrama de flujo

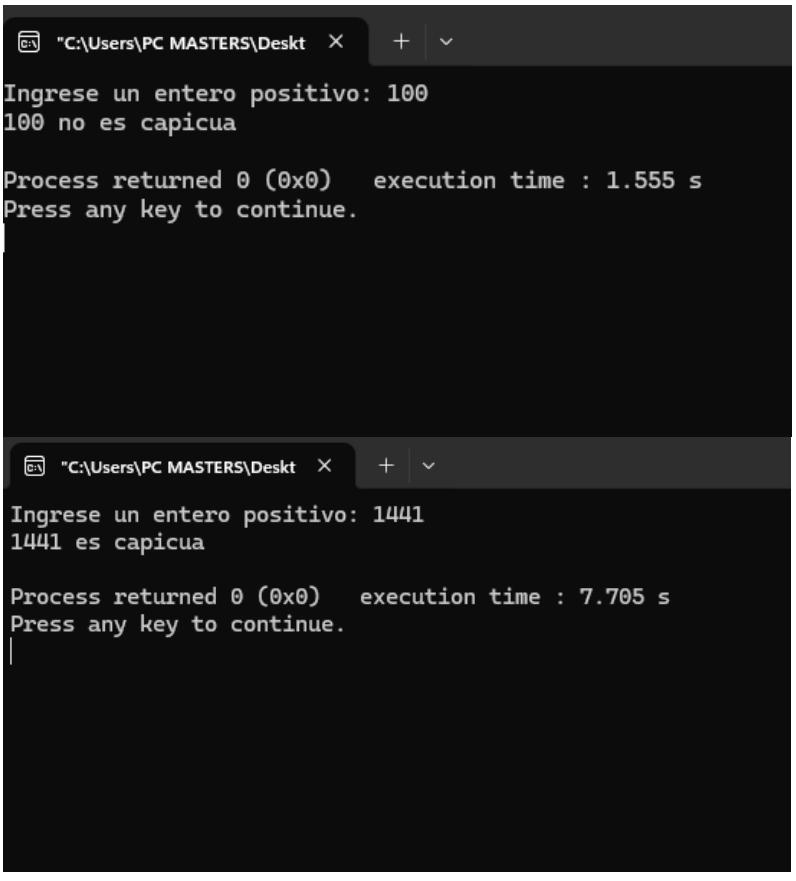


Prueba de escritorio

Paso	Condición	digito (MOD 10)	invertido	Salida (num_copy)
Inicio	—	—	—	1231
1	1231 > 0	1231 MOD 10 = 1	(0·10)+1 = 1	trunc(1231/10) = 123
2	123 > 0	123 MOD 10 = 3	(1·10)+3 = 13	trunc(123/10) = 12
3	12 > 0	12 MOD 10 = 2	(13·10)+2 = 132	trunc(12/10) = 1
4	1 > 0	1 MOD 10 = 1	(132·10)+1 = 1321	trunc(1/10) = 0
Fin	0 > 0 (No)	—	—	—
Comparación	1231 = 1321 (No)	—	—	1231 no lo es

Simulación del código





1.14 Máximo común divisor

Análisis del problema

- **Entrada:** Dos números enteros positivos a y b.
- **Proceso:** Evaluar que el valor de b sea diferente de cero (b > 0). Para sacar el residuo de la división entre a y b utilizamos a MOD b y guardamos este resultado. Posteriormente actualizamos los valores reemplazando a por el valor actual de b, y b por el residuo calculado.
- **Salida:** Mensaje que contenga la variable que tenga el M.C.D.

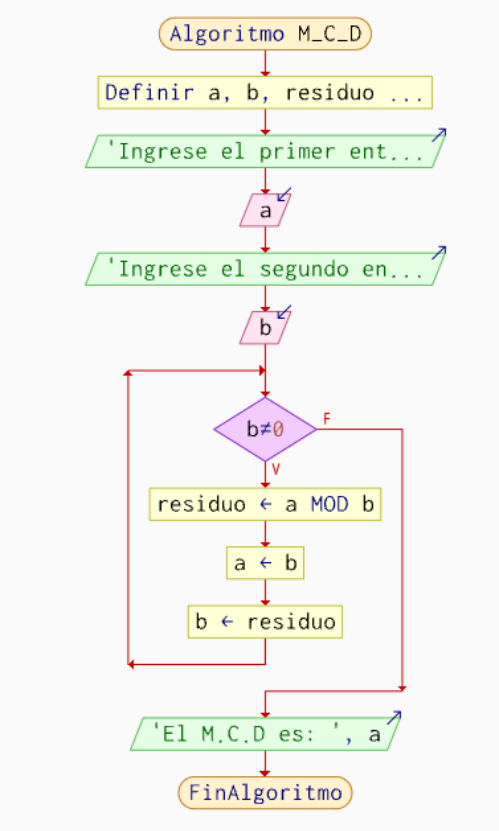
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Entrada # entero +	a	Var	Entero
Entrada # entero +	b	Var	Entero
Residuo	residuo	a MOD b	Entero

Seudocódigo

```
INICIO ALGORITMO MÁXIMO DIVISOR
  Definir a, b, residuo Como Entero
  Escribir "Ingrese el primer entero positivo: "
  Leer a
  Escribir "Ingrese el segundo entero positivo: "
  Leer b
  Mientras b <> 0 Hacer
    residuo <- a MOD b
    a <- b
    b <- residuo
  FinMientras
  Escribir "El M.C.D es: ", a
FIN ALGORITMO
```

Diagrama de flujo



Prueba de escritorio

Paso	Condición	residuo	a (final)	b (final)	Salida
Inicio	—	—	30	18	—
1	18 ≠ 0 (Sí)	30 MOD 18 = 12	18	12	—
2	12 ≠ 0 (Sí)	18 MOD 12 = 6	12	6	—
3	6 ≠ 0 (Sí)	12 MOD 6 = 0	6	0	—
4	0 ≠ 0 (No)	FIN	6	0	El M.C.D es: 6

Simulación del código

Start here | *EJERCICIO 10 FACTORIAL.c | *EJERCICIO 11 FIBONACCI.c | *EJERCICIO 12 SUMA

```c
#include <stdio.h>

int main()
{
    int a, b, resto;

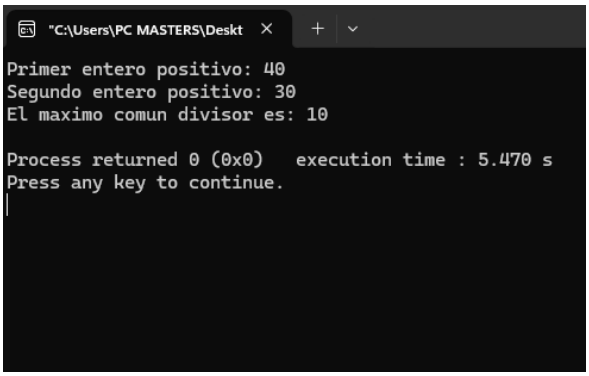
    printf("Primer entero positivo: ");
    scanf("%d", &a);
    printf("Segundo entero positivo: ");
    scanf("%d", &b);

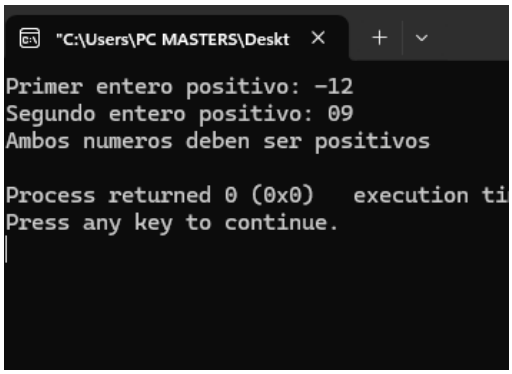
    if (a <= 0 || b <= 0)
    {
        printf("Ambos numeros deben ser positivos\n");
    }
    else
    {
        while (b != 0)
        {
            resto = a % b;
            a = b;
            b = resto;
        }
        printf("El maximo comun divisor es: %d\n", a);
    }
}
```

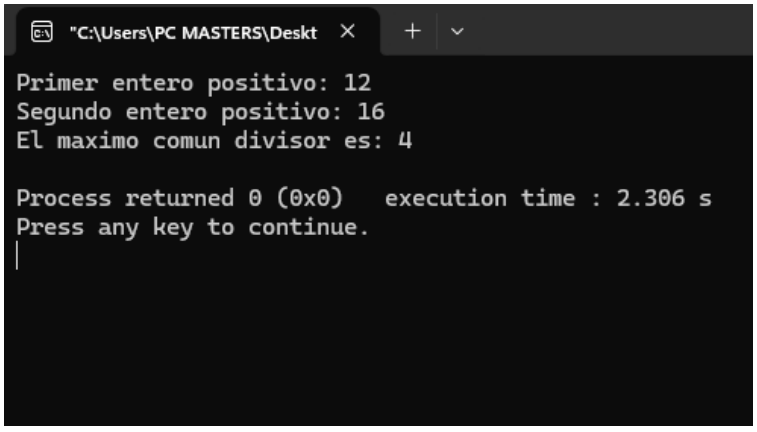
Logs & others

Code::Blocks | Search results | Cccc | Build log | Build messages

C:\Users\PC MASTERS\Desktop\PRIMER SEMESTRE\PROGRAMACION\EJERCICIO 14 MCD.c







1.15 Año bisiesto

Análisis del problema

- **Entrada:** Un número entero positivo, denominado abisiesto.
- **Proceso:** Evaluamos las condiciones que debería tener un año bisiesto mediante operadores lógicos. Un año bisiesto es divisible para 4 pero no para 100. Existe una excepción: si es divisible para 100, es posiblemente divisible para 400, y si es así se considera año bisiesto.
- **Salida:** Mensaje indicando si el año es bisiesto o no.

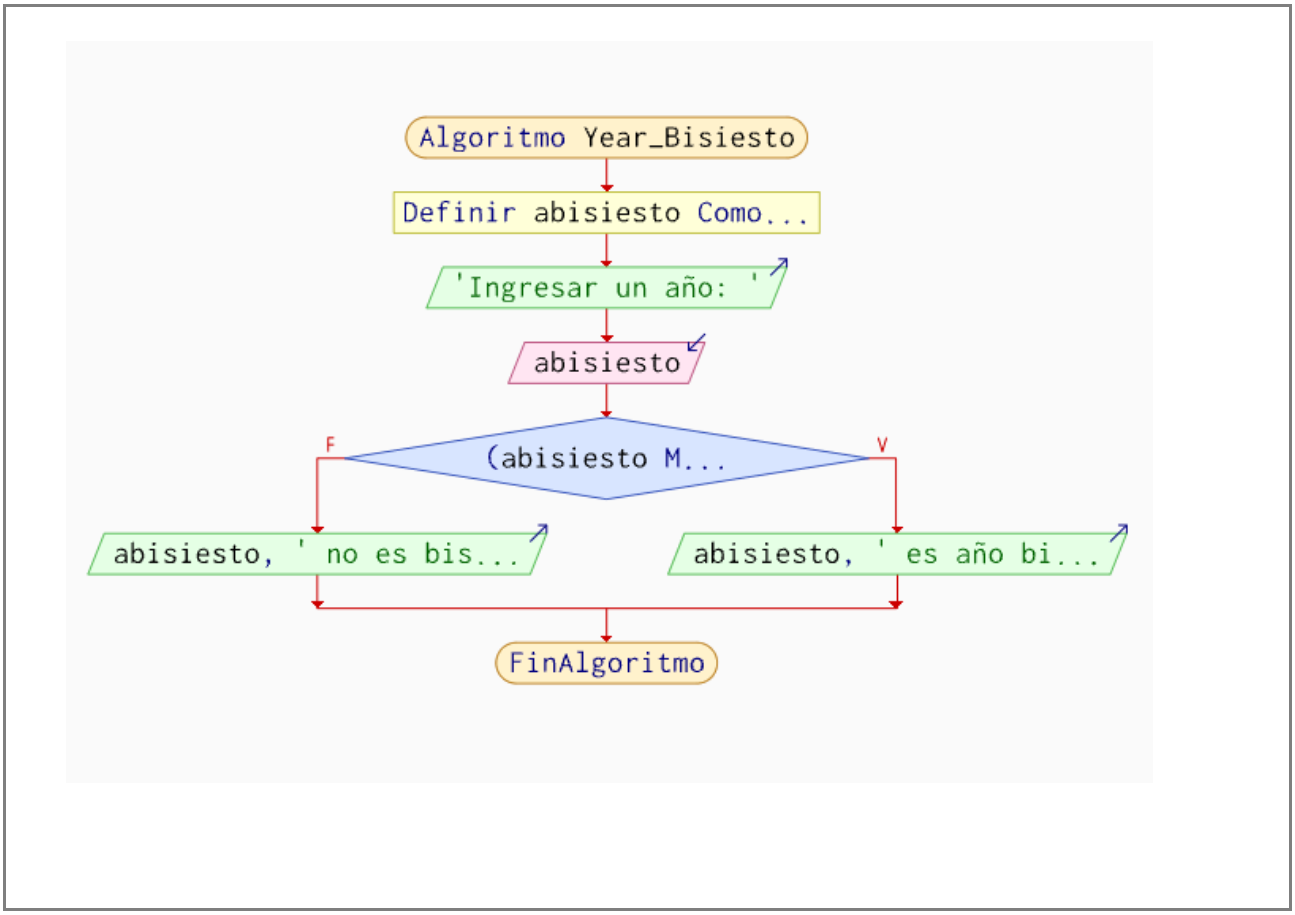
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Entrada año	abisiesto	Var	Entero

Seudocódigo

```
INICIO ALGORITMO AÑO BISIESTO
  Definir abisiesto Como Entero
  Escribir "Ingresar un año: "
  Leer abisiesto
  Si (abisiesto MOD 4 = 0 Y abisiesto MOD 100 <> 0) O (abisiesto MOD 400 = 0) Entonces
    Escribir abisiesto, " es año bisiesto"
  Sino
    Escribir abisiesto, " no es bisiesto"
  FinSi
FIN ALGORITMO
```

Diagrama de flujo



Prueba de escritorio

Año	abisiesto MOD 4 = 0	abisiesto MOD 100 ≠ 0	abisiesto MOD 400 = 0	Salida
2024	0 = 0 (V)	24 ≠ 0 (V)	24 = 0 (F)	Sí es bisiesto
1800	0 = 0 (V)	0 ≠ 0 (F)	200 = 0 (F)	No es bisiesto
2000	0 = 0 (V)	0 ≠ 0 (F)	0 = 0 (V)	Sí es bisiesto

Simulación del código

Start here | *EJERCICIO 10 FACTORIAL.c | *EJERCICIO 11 FIBONACCI.c | *EJERCICIO 12 SUMA DE DIGITOS.c

```c
#include <stdio.h>

int main()
{
    int year;

    printf("Ingrese un year: ");
    scanf("%d", &year);

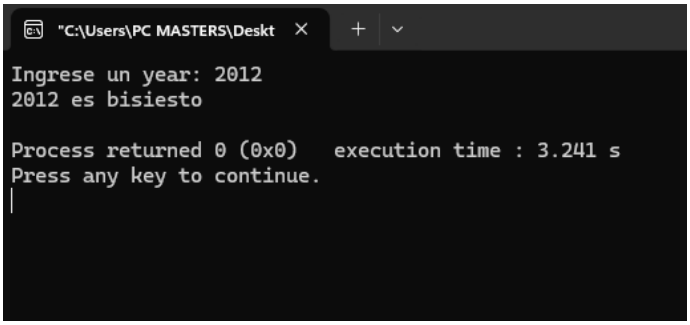
    if (year <= 0)
    {
        printf("El year debe ser positivo\n");
    }
    else
    {
        if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)
            printf("%d es bisiesto\n", year);
        else
            printf("%d no es bisiesto\n", year);
    }

}
```

Logs & others | Code::Blocks | Search results | Cccc | Build log | Build messages | CppCheck/Vera

C:\Users\PC MASTERS\Desktop\PRIMER SEMESTRE\PROGRAMACION\EJERCICIO 15 AÑO BISIESTO.c





1.16 Número perfecto

Análisis del problema

- **Entrada:** Un número entero positivo, denominado n.
- **Proceso:** Comenzamos con un acumulador suma = 0. Posteriormente usamos un ciclo para evaluar los posibles divisores desde i = 1 hasta llegar a n - 1. Si el residuo es 0 (n MOD i = 0), consideramos que es un divisor propio y lo sumamos al acumulador. Finalizamos el ciclo y comparamos la suma con n.
- **Salida:** Mensaje informando si n es perfecto o no.

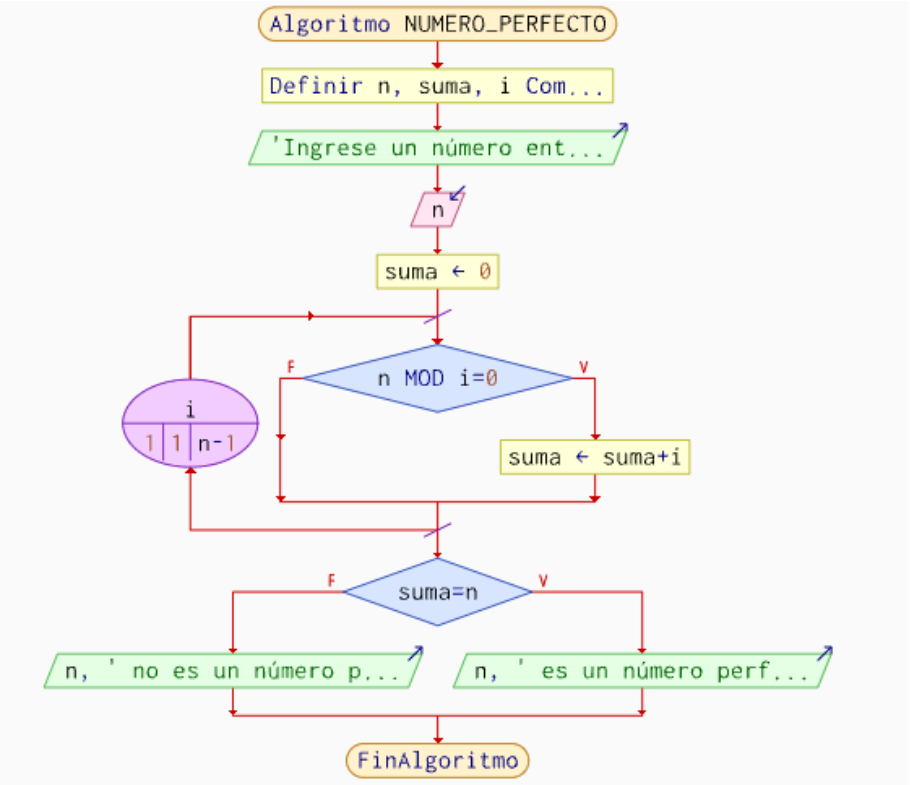
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Entrada # entero	n	Var	Entero
Acumulador suma	suma	0	Entero
Contador	i	1	Entero

Seudocódigo

```
INICIO ALGORITMO NÚMERO PERFECTO
  Definir n, suma, i Como Entero
  Escribir "Ingrese un número entero positivo: "
  Leer n
  suma <- 0
  Para i <- 1 Hasta n-1 Con Paso 1 Hacer
    Si n MOD i = 0 Entonces
      suma <- suma + i
    FinSi
  FinPara
  Si suma = n Entonces
    Escribir n, " es un número perfecto"
  Sino
    Escribir n, " no es un número perfecto"
  FinSi
FIN ALGORITMO
```

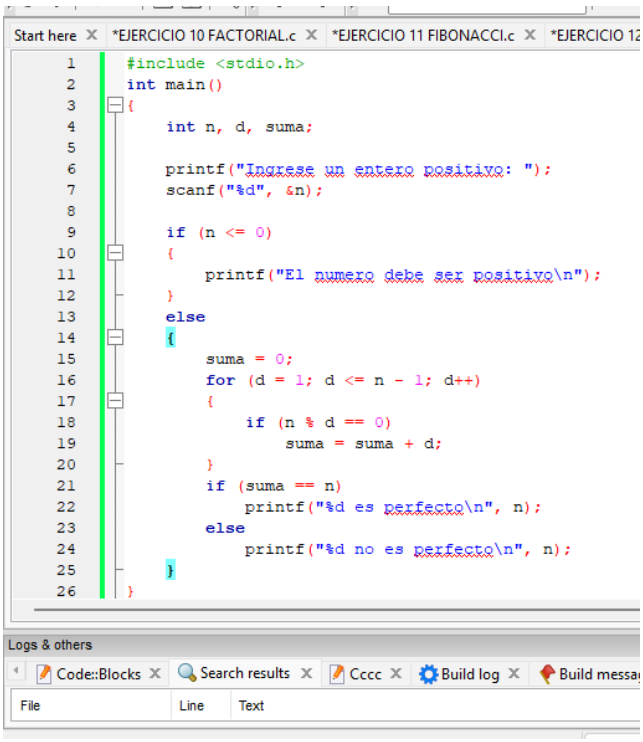
Diagrama de flujo

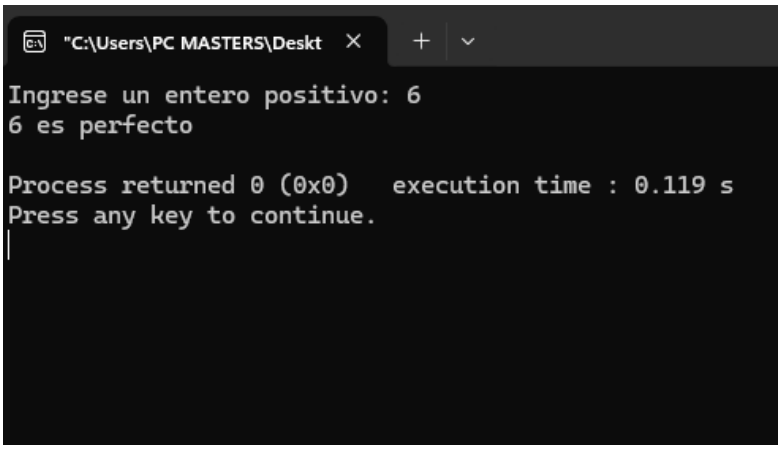


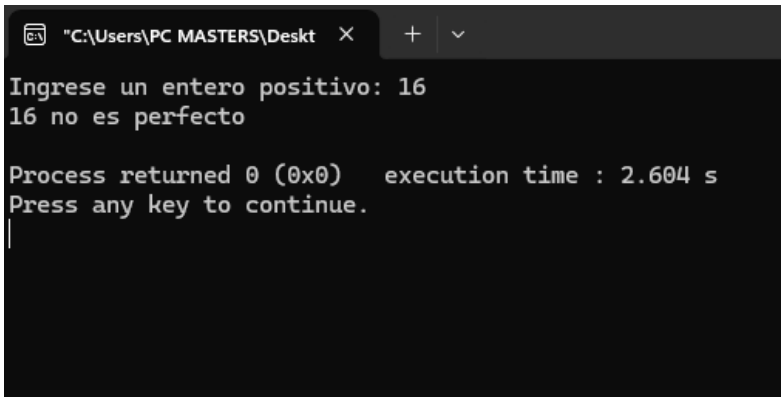
Prueba de escritorio

Paso	i ≤ n-1	n MOD i = 0	suma	i (nuevo)
Inicio	—	—	0	1
1	1 ≤ 5	6 MOD 1 = 0 (Sí)	0+1 = 1	2
2	2 ≤ 5	6 MOD 2 = 0 (Sí)	1+2 = 3	3
3	3 ≤ 5	6 MOD 3 = 0 (Sí)	3+3 = 6	4
4	4 ≤ 5	6 MOD 4 = 2 (No)	6	5
5	5 ≤ 5	6 MOD 5 = 1 (No)	6	6
6	6 ≤ 5 (Falso) → FIN CICLO	—	6	—
Resultado	suma = n (6 = 6)	—	—	6 es un número perfecto

Simulación del código







1.17 Conversión de decimal a binario

Análisis del problema

- **Entrada:** Un número entero positivo, denominado num.
- **Proceso:** A nuestro num entero necesitamos dividirlo sucesivamente para 2 y guardar los restos, para que mediante pasos matemáticos multipliquemos cada nuevo residuo por una variable posición y lo sumemos a un acumulador binario. El resultado del residuo se va colocando de frente a los dígitos anteriores, mientras num > 0.
- **Salida:** Mostrar la variable binaria que contiene la secuencia final tanto de ceros como de unos.

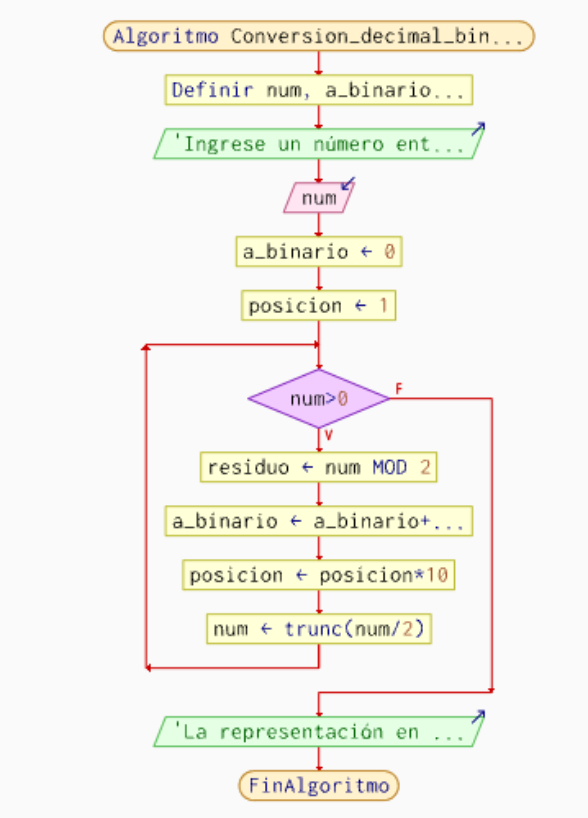
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Entrada # entero	num	Var	Entero
Acumulador	a_binario	0	Entero
Posición decimal	posicion	1	Entero
Residuo	residuo	0	Entero

Seudocódigo

```
INICIO ALGORITMO DECIMAL A BINARIO
  Definir num, a_binario, posicion, residuo Como Entero
  Escribir "Ingrese un número entero positivo en base 10"
  Leer num
  a_binario <- 0
  posicion <- 1
  Mientras num > 0 Hacer
    residuo <- num MOD 2
    a_binario <- a_binario + (residuo * posicion)
    posicion <- posicion * 10
    num <- trunc(num / 2)
  FinMientras
  Escribir "La representación en binario es: ", a_binario
FIN ALGORITMO
```

Diagrama de flujo

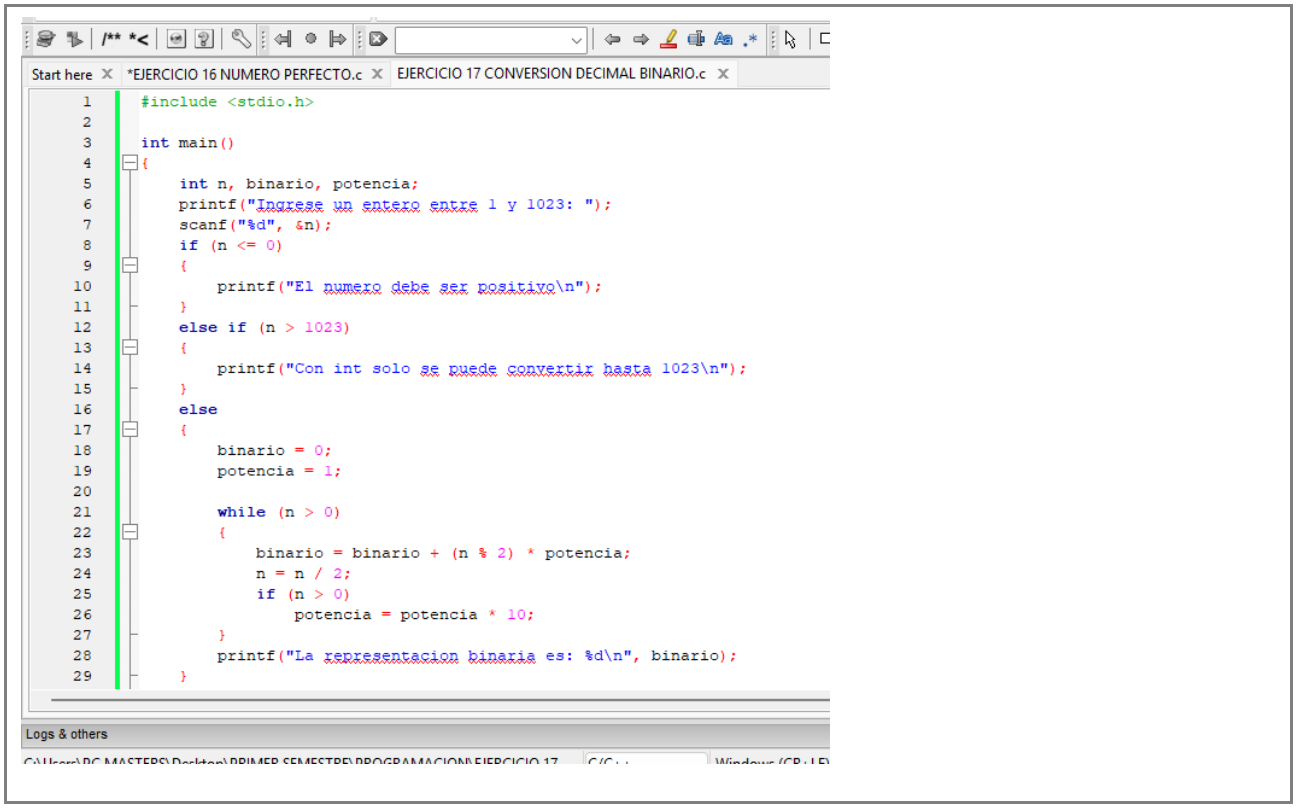


Prueba de escritorio

Paso	num	residuo	a_binario	posicion × 10	trunc(num/2)
Inicio	13	—	0	posicion = 1	—
1	13 > 0	13 MOD 2 = 1	0 + (1·1) = 1	1·10 = 10	6
2	6 > 0	6 MOD 2 = 0	1 + (0·10) = 1	10·10 = 100	3
3	3 > 0	3 MOD 2 = 1	1 + (1·100) = 101	100·10 = 1000	1
4	1 > 0	1 MOD 2 = 1	101 + (1·1000) = 1101	1000·10 = 10000	0
Fin	0 > 0 (No)	—	1101	—	—

Salida en pantalla: La representación en binario es: 1101

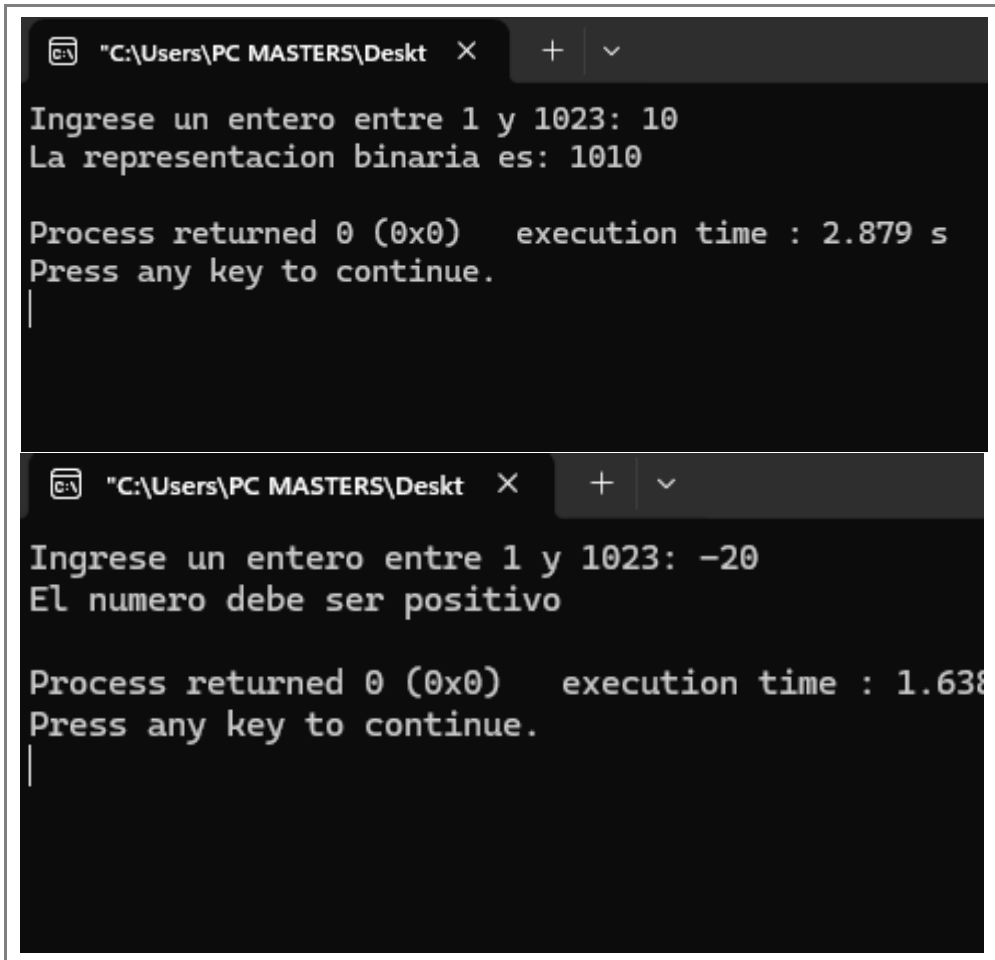
Simulación del código




```c
#include <stdio.h>

int main()
{
    int n, binario, potencia;
    printf("Ingrese un entero entre 1 y 1023: ");
    scanf("%d", &n);
    if (n <= 0)
    {
        printf("El numero debe ser positivo\n");
    }
    else if (n > 1023)
    {
        printf("Con int solo se puede convertir hasta 1023\n");
    }
    else
    {
        binario = 0;
        potencia = 1;

        while (n > 0)
        {
            binario = binario + (n % 2) * potencia;
            n = n / 2;
            if (n > 0)
                potencia = potencia * 10;
        }
        printf("La representacion binaria es: %d\n", binario);
    }
```



1.18 Triángulo de asteriscos

Análisis del problema

- **Entrada:** Un número entero, denominado n, que representa la cantidad de filas.
- **Proceso:** Comenzamos con un ciclo externo para cada fila i de 1 a n. Dentro colocaremos un ciclo interno que escribe j asteriscos en la misma línea. Al culminar el proceso interno, se salta de línea.
- **Salida:** Un triángulo rectángulo de n filas representado por asteriscos.

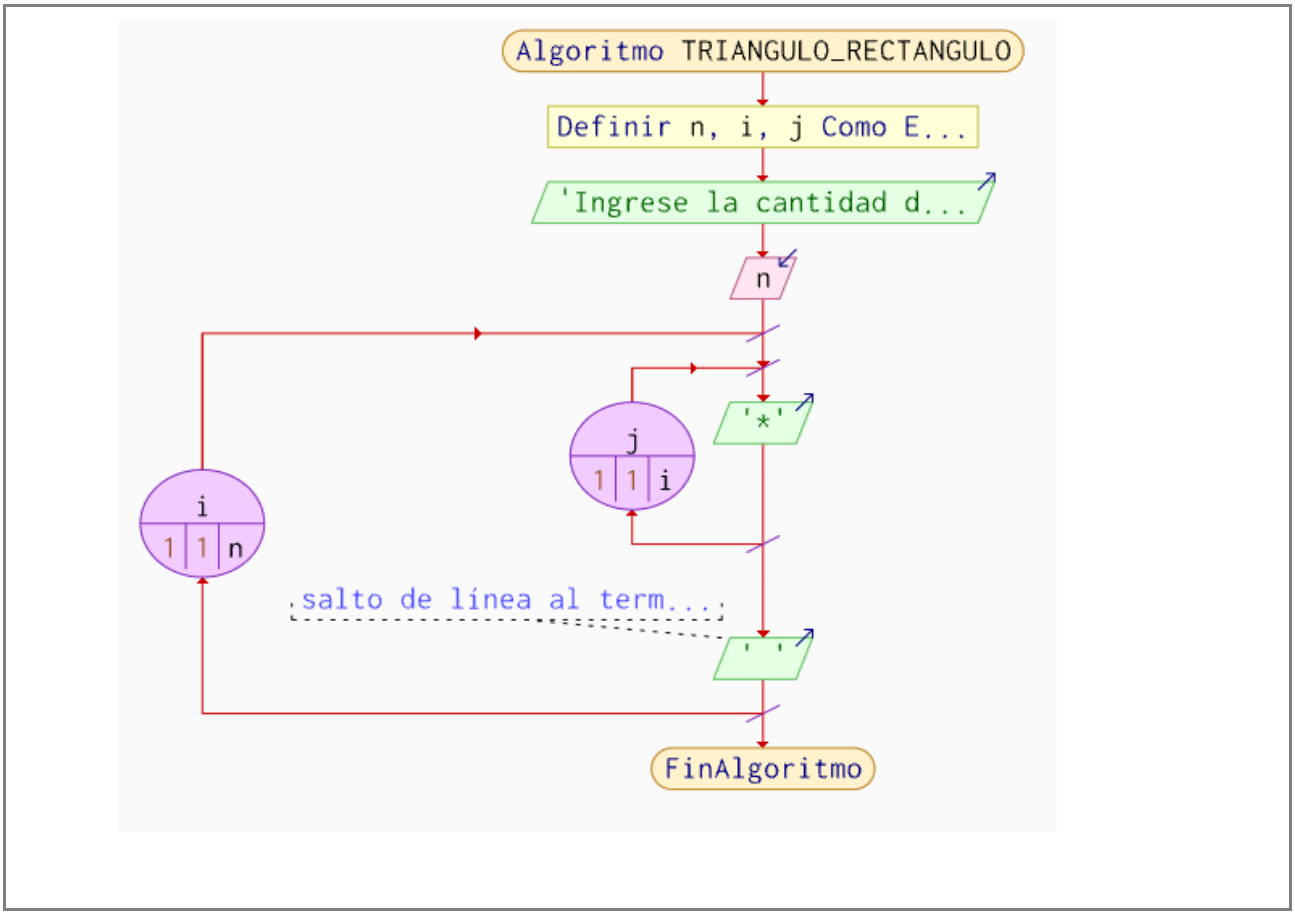
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Entrada # entero	n	Var	Entero
C. Fila actual	i	Var / 1	Entero
C. Asterisco	j	Var / 1	Entero

Seudocódigo

```
INICIO ALGORITMO TRIÁNGULO RECTÁNGULO
  Definir n, i, j Como Entero
  Escribir "Ingrese la cantidad de filas mayores o iguales a 1: "
  Leer n
  Para i <- 1 Hasta n Con Paso 1 Hacer
    Para j <- 1 Hasta i Con Paso 1 Hacer
      Escribir Sin Saltar "*"
    FinPara
    Escribir " "   // salto de línea al terminar la fila
  FinPara
FIN ALGORITMO
```

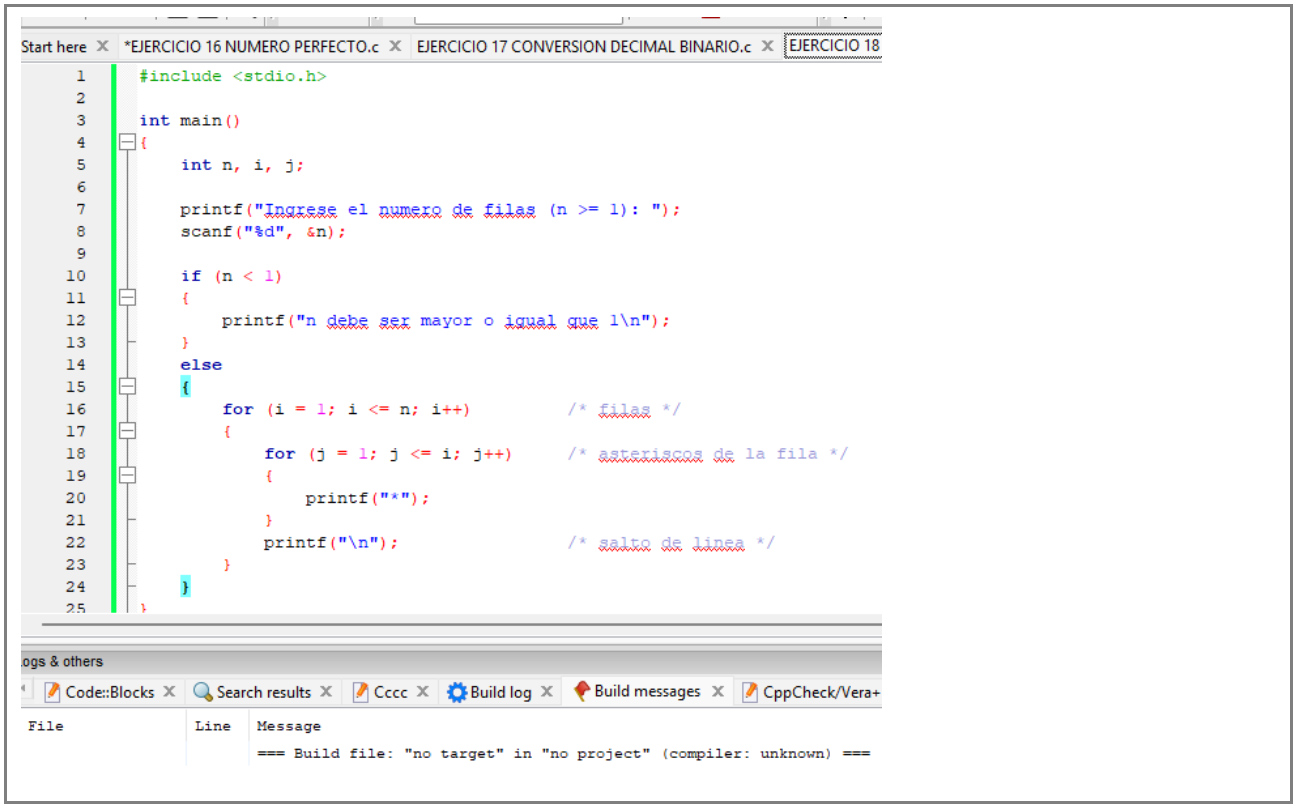
Diagrama de flujo



Prueba de escritorio

Ciclo (i)	i ≤ n	j	j ≤ i	Escribe	Nuevo j	Nuevo i
1	1 ≤ 3 (Sí)	1	1 ≤ 1 (Sí)	*	2	—
		2	2 ≤ 1 (No)	salto de línea	—	i = 1+1 = 2
2	2 ≤ 3 (Sí)	1	1 ≤ 2 (Sí)	*	2	—
		2	2 ≤ 2 (Sí)	* *	3	—
		3	3 ≤ 2 (No)	salto de línea	—	i = 2+1 = 3
3	3 ≤ 3 (Sí)	1	1 ≤ 3 (Sí)	*	2	—
		2	2 ≤ 3 (Sí)	* *	3	—
		3	3 ≤ 3 (Sí)	* * *	4	—
		4	4 ≤ 3 (No)	salto de línea	—	i = 3+1 = 4
Fin	4 ≤ 3 (No)			Fin algoritmo		

Simulación del código



```
Start here | *EJERCICIO 16 NUMERO PERFECTO.c | EJERCICIO 17 CONVERSION DECIMAL BINARIO.c | EJERCICIO 18

#include <stdio.h>

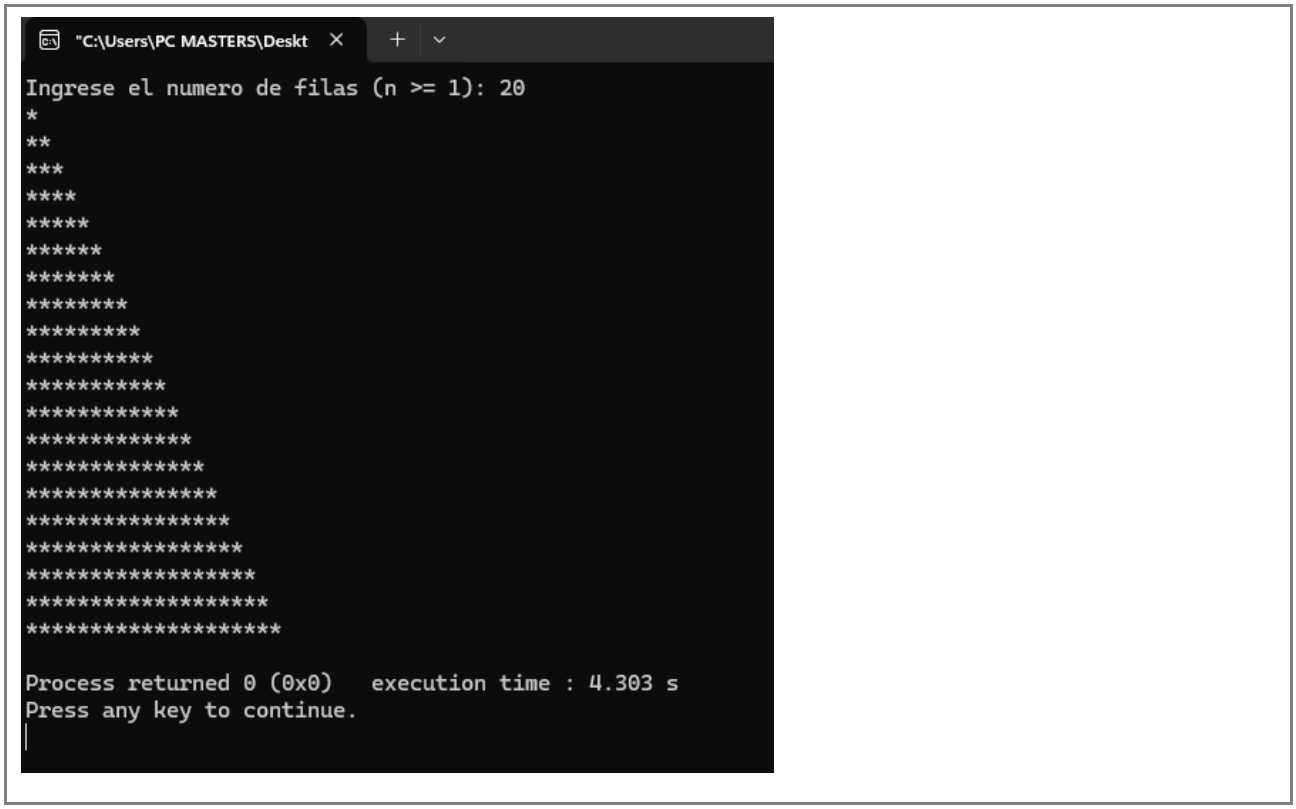
int main()
{
    int n, i, j;

    printf("Ingrese el numero de filas (n >= 1): ");
    scanf("%d", &n);

    if (n < 1)
    {
        printf("n debe ser mayor o igual que 1\n");
    }
    else
    {
        for (i = 1; i <= n; i++)          /* filas */
        {
            for (j = 1; j <= i; j++)      /* asteriscos de la fila */
            {
                printf("*");
            }
            printf("\n");                 /* salto de linea */
        }
    }
}

ogs & others
Code::Blocks | Search results | Cccc | Build log | Build messages | CppCheck/Vera+

File | Line | Message
     |      | === Build file: "no target" in "no project" (compiler: unknown) ===
```



```
"C:\Users\PC MASTERS\Deskt
Ingrese el numero de filas (n >= 1): 20
*
**
***
****
*****
******
*******
********
*********
**********
***********
************
*************
**************
***************
****************
*****************
******************
*******************
********************

Process returned 0 (0x0)   execution time : 4.303 s
Press any key to continue.
```

1.19 Segundo mayor de una secuencia

Análisis del problema

- **Entradas:** Secuencia de números enteros positivos.
- **Procesos:** Mayor y segundo parten de 0, por ende no tienen valor. Si x > mayor: el mayor anterior pasa a ser el segundo y x pasa a ser el mayor; si no, y x > segundo (y x < mayor), x pasa a ser el segundo. Si al terminar el segundo = 0, se informa que no hay suficientes valores.
- **Salidas:** El número mayor de la secuencia, el segundo mayor de la secuencia y un mensaje de error si no se tienen al menos 2 números válidos.
- **Restricciones:** Los datos son enteros positivos y el 0 termina la lectura.
- **Valores repetidos:** Si el número más alto aparece más de una vez, el primer y el segundo puesto tomarán ese valor, ya que el segundo valor más alto disponible sigue siendo ese número.

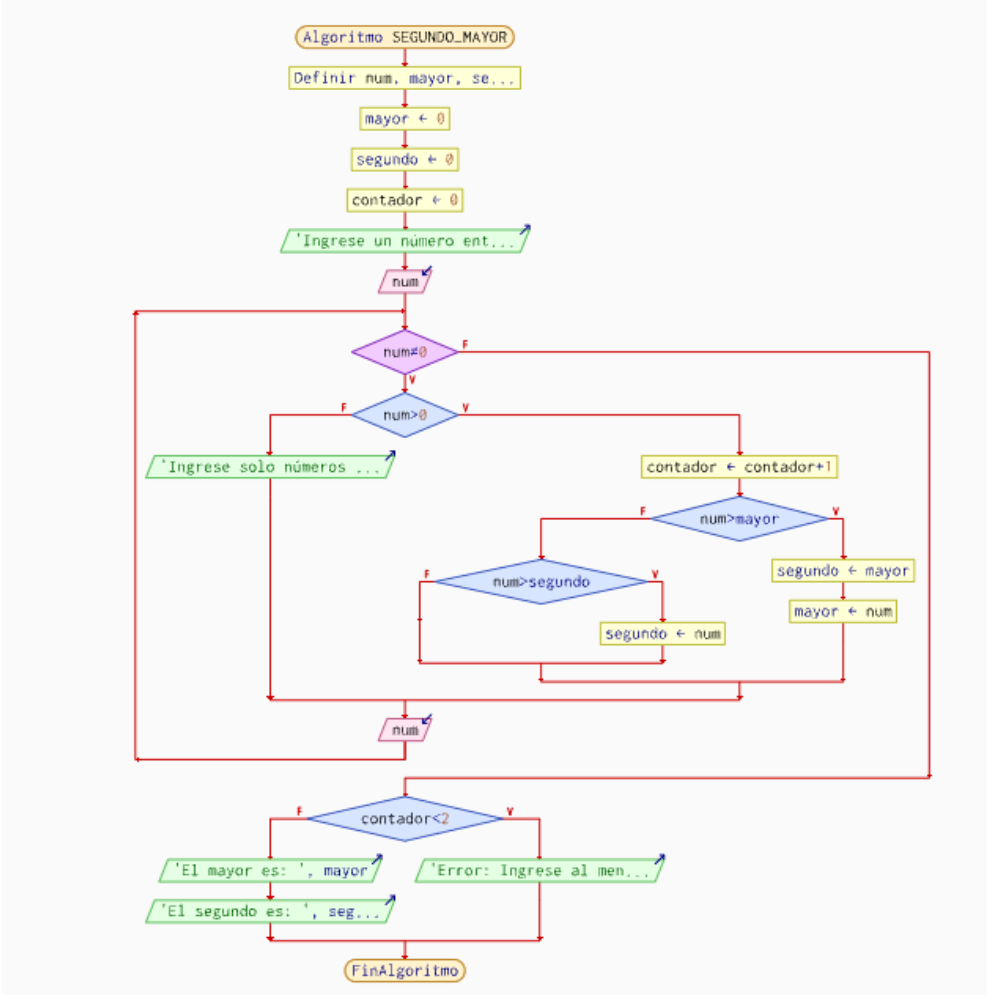
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Entrada # entero	num	Ninguno	Entero
Almacena el valor mayor	mayor	0	Entero
Almacena el segundo valor mayor	segundo	0	Entero
Cantidad de # válidos	contador	0	Entero

Seudocódigo

```
INICIO ALGORITMO SEGUNDO MAYOR
  Definir num, mayor, segundo, contador Como Entero
  mayor <- 0
  segundo <- 0
  contador <- 0
  Escribir "Ingrese un número entero positivo (coloque 0 para terminar)"
  Leer num
  Mientras num <> 0 Hacer
    Si num > 0 Entonces
      contador <- contador + 1
      Si num > mayor Entonces
        segundo <- mayor
        mayor <- num
      Sino
        Si num > segundo Entonces
          segundo <- num
        FinSi
      FinSi
    Sino
      Escribir "Ingrese solo números positivos"
    FinSi
    Leer num
  FinMientras
  Si contador < 2 Entonces
    Escribir "Error: Ingrese al menos 2 números válidos"
  Sino
    Escribir "El mayor es: ", mayor
    Escribir "El segundo es: ", segundo
  FinSi
FIN ALGORITMO
```

Diagrama de flujo



Prueba de escritorio

Paso	num	Condición	mayor	segundo	contador	Salida
Inicio	—	—	0	0	0	—
1	12	12 > 0 (V)	12	0	1	El antiguo mayor (0) pasa a ser el segundo; 12 es el nuevo mayor
2	17	17 > 12 (V)	17	12	2	El antiguo mayor (12) pasa a ser el segundo
3	0	num <> 0 (F)	17	12	2	Termina el ciclo
Fin	—	contador < 2 (F)	17	12	2	El mayor es: 17 / El segundo es: 12

Simulación del código

here | *EJERCICIO 16 NUMERO PERFECTO.c | EJERCICIO 17 CONVERSION DECIMAL BINARIO.c

```c
#include <stdio.h>

int main()
{
    int x, mayor = 0, segundo = 0;

    printf("Entero positivo (0 termina): ");
    scanf("%d", &x);

    while (x != 0)
    {
        if (x > mayor)
        {
            segundo = mayor;
            mayor = x;
        }
        else if (x > segundo && x < mayor)
        {
            segundo = x;
        }

        printf("Otro entero positivo (0 termina): ");
        scanf("%d", &x);
    }
    if (segundo == 0)
        printf("Se necesitan al menos dos valores distintos\n");
    else
        printf("Mayor = %d, segundo mayor = %d\n", mayor, segundo);

    return 0;
}
```

s & others

Code::Blocks | Search results | Cccc | Build log | Build messages | CppCheck/Vera++

File	Line	Message
		=== Build file: "no target" in "no project" (compiler: unknown) ===

"C:\Users\PC MASTERS\Deskt

```
Entero positivo (0 termina): 10
Otro entero positivo (0 termina):
80
Otro entero positivo (0 termina): 90
Otro entero positivo (0 termina): 23
Otro entero positivo (0 termina): 32
Otro entero positivo (0 termina): 0
Mayor = 90, segundo mayor = 80

Process returned 0 (0x0)   execution time : 11.916 s
Press any key to continue.
```

1.20 Adivina el número

Análisis del problema

- **Entradas:** Los intentos que el usuario escribe (serie).
- **Proceso:** El programa fija un número secreto. En cada vuelta se lee el intento, se cuenta y se compara: si es menor que el número secreto dice "más alto" y viceversa; si es igual dice "acertaste". El ciclo se repite hasta que intento = secreto.
- **Salida:** Pista después de cada intento y, al final, la cantidad de intentos.

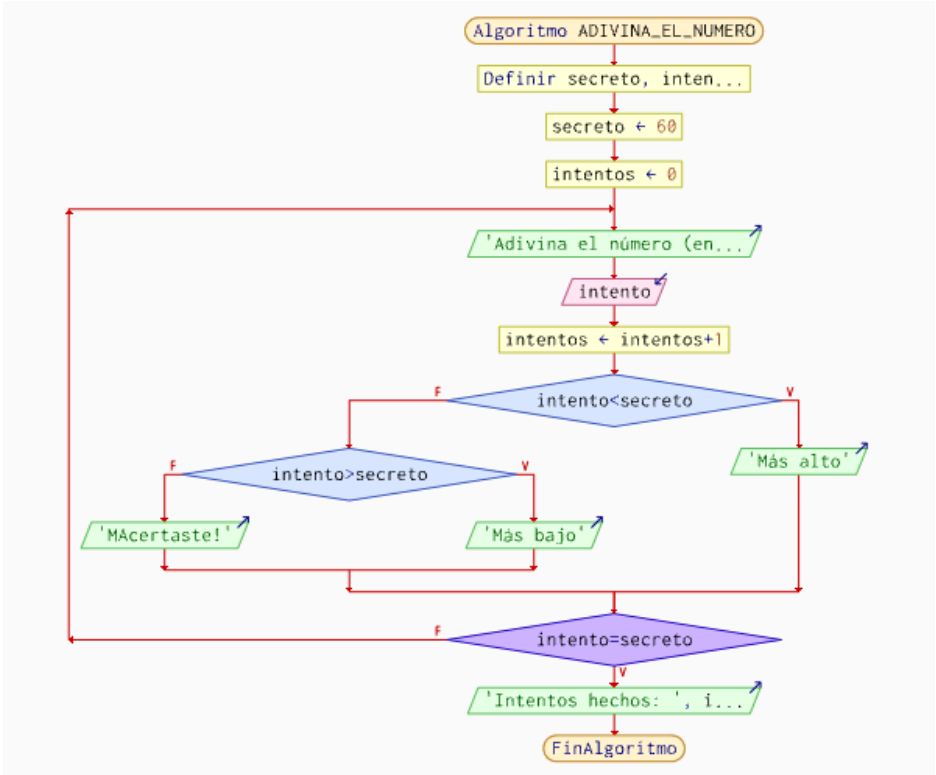
Tabla de objetos

Objeto	Nombre	Valor	Tipo
Número Secreto	secreto	Cte	Entero
Intento del usuario	intento	Var	Entero
Cantidad de intentos	intentos	Var	Entero
Contador inicial	0	Cte	Entero
Incremento contador	1	Cte	Entero

Seudocódigo

```
INICIO ALGORITMO ADIVINA EL NÚMERO
  Definir secreto, intento, intentos Como Entero
  secreto <- 60
  intentos <- 0
  Repetir
    Escribir "Adivina el número (entre 1 y 100): "
    Leer intento
    intentos <- intentos + 1
    Si intento < secreto Entonces
      Escribir "Más alto"
    Sino
      Si intento > secreto Entonces
        Escribir "Más bajo"
      Sino
        Escribir "¡Acertaste!"
      FinSi
    FinSi
  Hasta Que intento = secreto
  Escribir "Intentos hechos: ", intentos
FIN ALGORITMO
```

Diagrama de flujo



Prueba de escritorio

Intento (n.º)	Número	Comparación	Pista	Salida / intentos
1	75	75 > 60	Más bajo	1
2	25	25 < 60	Más alto	2
3	60	60 = 60	¡Acertaste!	3
Fin del juego				Intentos hechos: 3

Simulación del código

```c
#include <stdio.h>

int main()
{
    const int secreto = 60;
    int intento, intentos = 0;

    do
    {
        printf("Adivina el numero (entre 1 y 100): ");
        scanf("%d", &intento);
        intentos++;

        if (intento < secreto)
            printf("mas alto\n");
        else if (intento > secreto)
            printf("mas bajo\n");
        else
            printf("acertaste!\n");
    }
    while (intento != secreto);

    printf("Intentos usados: %d\n", intentos);
}
```

Logs & others

Code::Blocks | Search results | Cccc | Build log | Build messages | CppCh

File	Line	Message

```
"C:\Users\PC MASTERS\Deskt
Adivina el numero (entre 1 y 100): 10
mas alto
Adivina el numero (entre 1 y 100): 80
mas bajo
Adivina el numero (entre 1 y 100): 40
mas alto
Adivina el numero (entre 1 y 100): 60
acertaste!
Intentos usados: 4

Process returned 0 (0x0)   execution time : 8.684 s
Press any key to continue.
```